

# **“Society/Apartment Management Portal”**

A Project Report Submitted in the partial fulfillment of the requirements for the award of the degree of

## **BACHELOR OF TECHNOLOGY**

**In**

**DEPARTMENT OF COMPUTER SCIENCE ENGINEERING**

**&**

**DEPARTMENT OF COMPUTER SCIENCE AND INFORMATION TECHNOLOGY**

**By**

**S . SARVESWARA REDDY :2520030049**

**GUNNAM ROHITH :2520030220**

**VEMULA GAUTAM :2520030316**

Under the Esteemed Guidance of

**Dr. Satish Kumar**

**Assistant Professor**

**Department of Computer Science and Engineering**



## **Koneru Lakshmaiah Education Foundation**

(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)

Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.

Phone No: 7815926816, [www.klh.edu.in](http://www.klh.edu.in)

**K L (Deemed to be) University  
DEPARTMENT OF COMPUTER SCIENCE ENGINEERING**

**&**

**DEPARTMENT OF COMPUTER SCIENCE AND INFORMATION TECHNOLOGY**



### **Declaration**

The Project Report entitled **“Society/Apartment Management Portal”** is a record of Bonafide work of **S.SARVESH REDDY - 2520030049 , GUNNAM ROHITH - 2520030220 , VEMULA GAUTAM - 2520030316** submitted in partial fulfillment for the award of B. Tech in Computer Science and Engineering (or) Computer Science and Information Technology to the K L University. The results embodied in this report have not been copied from any other departments/University/Institute.

**S.SARVESH REDDY - 2520030049**

**GUNNAM ROHITH - 2520030220**

**VEMULA GAUTAM - 2520030316**

**K L (Deemed to be) University**

**DEPARTMENT OF COMPUTER SCIENCE ENGINEERING**

**&**

**DEPARTMENT OF COMPUTER SCIENCE AND INFORMATION TECHNOLOGY**



## **CERTIFICATE**

This is certify that the mini project based report entitled **“Society/Apartment Management Portal”** is a bonafide work done and submitted by **S.SARVESH REDDY - 2520030049 , GUNNAM ROHITH - 2520030220 , VEMULA GAUTAM - 2520030316** in partial fulfillment of the requirements for the award of the degree of **BACHELOR OF TECHNOLOGY** in Department of Computer Science Engineering, K L (Deemed to be University), during the academic year **2024-2025**.

**Signature of the Guide**

**Signature of the Course Coordinator**

**Signature of the HOD**

## ACKNOWLEDGEMENT

The success in this project would not have been possible but for the timely help and guidance rendered by many people. Our wish to express my sincere thanks to all those who has assisted us in one way or the other for the completion of my project.

Our greatest appreciation to my Course Coordinator **Dr Yerragudipadu Subbarayudu**, and my guide **Dr Y Prasanthi**, Department of Computer Science which cannot be expressed in words for his/her tremendous support, encouragement and guidance for this project.

We express our gratitude to **Dr. P Venkateshwara Roa (CSE)/Dr. Kaja Shareef (CSIT)**, Head of the **Department for Computer Science Engineering/ Department for Computer Science and Information Technology** for providing us with adequate facilities, ways and means by which we are able to complete this project-based Lab.

We thank all the members of teaching and non-teaching staff members, and also who have assisted me directly or indirectly for successful completion of this project. Finally, I sincerely thank my parents, friends and classmates for their kind help and cooperation during my work.

**S.SARVESH REDDY - 2520030049**

**GUNNAM ROHITH - 2520030220**

**VEMULA GAUTAM - 2520030316**

## **Abstract**

- The Society/Apartment Management Portal (Haven Society Management) is a web-based application designed to simplify and digitize the day-to-day management activities of residential apartments and housing societies. In many apartments, activities such as maintenance billing, payment tracking, complaint handling, visitor entry, notices, and amenity booking are managed manually or through separate registers, spreadsheets, and chat groups. This results in duplicated records, delayed communication, and difficulty in tracking dues and requests.
- The proposed system provides a centralized, responsive platform organized into ten modules—Flats, Residents, Visitors, Staff, Maintenance Bills, Payments, Complaints, Notices, Amenities, and Amenity Bookings. It supports eight user roles (Admin, Committee, Treasurer, Secretary, Resident, Security, Housekeeping, and Maintenance Staff), each with its own dashboard, navigation, and permitted actions. Administrators get an analytics dashboard with occupancy, dues, and complaint charts; treasurers create bills and record payments; residents view their own dues and payment history, raise complaints, register visitors, and book amenities; and security staff maintain a digital visitor register.
- The frontend is built with React 19 and Vite, using React Router, Recharts, and Lucide React, with role-based and record-level access rules defined in a central configuration. The data model is a normalized ten-table MySQL database (society\_management), and all data operations pass through an asynchronous service layer designed for integration with a Node.js/Express REST API. The application was verified with ESLint, a production build, and 23 automated browser test cases. Overall, the project provides an organized, convenient, and centralized solution for managing residential society operations.

# Index

| S. No. | Chapters                 | Topics                                                                                                                                                                              | Page.no |
|--------|--------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------|
|        |                          | Acknowledgement                                                                                                                                                                     | 4       |
|        |                          | Abstract                                                                                                                                                                            | 5 8-    |
| 1      | Introduction             | 1.1 Background of the project<br>1.2 Problem statement<br>1.3 Scope of the project<br>1.4 Objective(s)<br>1.5 Importance of the application<br>1.6 Target users/audience            | 14      |
| 2      | System Requirements      | 2.1 Hardware requirements<br>2.2 Software requirements<br>2.3 Development tools and frameworks                                                                                      | 15-17   |
| 3      | Technology Stack         | 3.1 Front-end (React.js, HTML/CSS)<br>3.2 Back-end (Node.js / Express)<br>3.3 Database (MySQL)<br>3.4 Version Control (Git, GitHub)<br>3.5 APIs and External services               | 18-22   |
| 4      | System Architecture      | 4.1 High-level architecture diagram<br>4.2 Description of each layer (frontend, backend, database)<br>4.3 Deployment architecture (cloud, local, CI/CD pipelines)<br>5.1 ER Diagram | 23-26   |
| 5      | Design                   | 5.2 Database schema<br>6.1 Module-wise implementation                                                                                                                               | 27-31   |
| 6      | Implementation           | 6.2 Front-end logic (UI rendering, state management)<br>6.3 API integration<br>6.4 Authentication & Authorization<br>7.1 List of main features                                      | 32-39   |
| 7      | Features                 | 7.2 How each feature works (user perspective + technical description)<br>8.1 Postman testing (frontend/backend)                                                                     | 40-43   |
| 8      | Testing                  | 8.2 Integration testing<br>8.3 Tools used for testing<br>8.4 Test cases and results<br>9.1 Steps to deploy                                                                          | 44-48   |
| 9      | Deployment               | 9.2 Environment configuration<br>9.3 Hosting of frontend and backend<br>10.1 Issues faced during development<br>10.2 Solutions applied                                              | 49-52   |
| 10     | Challenges & Limitations | 10.3 Current limitations or known bugs                                                                                                                                              | 53-56   |

|             |                     |                                                                                                                                                              |                 |
|-------------|---------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------|
| 11          | Future Enhancements | 11.1 Planned features<br>11.2 Possible integrations or optimizations                                                                                         | 57-59           |
| 12          | Conclusion          | 12.1 Summary of the project<br>12.2 What was achieved<br>12.3 Skills learned during development                                                              | 60-62           |
| 1           | References          | - Books, official documentation, tutorials, and standards used<br><del>- Screenshots of the app</del>                                                        | 63-             |
| 3<br>1<br>4 | Appendices          | - Sample code snippets<br>- Installation/setup instructions<br>- User manual or guide<br>- Geo Tag photos with guide<br>- Review forms with guide signatures | 64<br>65-<br>85 |

## CHAPTER -1 INTRODUCTION

### 1.1 Background of the Project

Residential societies are complex communities in which a large number of people, properties, services, financial activities, and administrative operations have to be managed on a continuous basis. A residential society is not limited to the physical management of flats and buildings; it also involves maintaining resident information, monitoring visitors, coordinating security and housekeeping staff, generating maintenance bills, recording payments, handling complaints, publishing notices, managing common facilities, and maintaining amenity booking schedules. As the size of a residential community increases, the volume of information generated through these activities also increases. Managing such information manually through notebooks, paper registers, spreadsheets, or separate communication platforms can become difficult because the information is distributed across different locations and may not always remain synchronized. The Society Management Portal, also referred to in the project design as Haven Society Management, has been designed as a centralized residential community management system to organize these activities within a structured digital environment. The project design specifically defines the system as a residential society operations and community management system covering flats, residents, visitors, staff, maintenance billing, payments, complaints, notices, amenities, and amenity bookings.

The fundamental idea behind the project is to create a single application through which important society operations can be represented, accessed, and managed in an organized manner. In a conventional society management environment, information about a flat may be stored separately from information about its residents, while visitor records may be maintained by security personnel and maintenance information may be handled by the finance or administrative team. Complaints may be received through informal communication channels, and amenity reservations may be maintained using separate registers or spreadsheets. Such an arrangement makes it difficult to obtain a complete and consistent view of society activities. The proposed system addresses this issue by organizing the major operational areas into clearly defined modules and connecting related records through a relational data model. For example, a resident can be associated with a particular flat, a visitor can be associated with a destination flat, a maintenance bill can be associated with a flat, a payment can be associated with both a bill and a resident, and an amenity booking can be associated with both a resident and an amenity. The project therefore treats society management not as a collection of independent functions but as an interconnected information system in which changes and records in one area can have relevance to another.

The project has been designed using a modern web application approach. The frontend is developed using React and Vite, with HTML and CSS forming the basic web presentation technologies. React

provides the component-based structure required to develop reusable user-interface elements, while Vite provides the development and build environment. The project design also identifies React Router, Recharts, and Lucide React as application libraries. The intended database is MySQL, with the database named `society_management`. The architecture places a context and service layer between the React interface and the future backend API, thereby avoiding direct communication between the frontend and the database. This separation is important from a software engineering perspective because it allows the presentation layer, application logic, and data storage responsibilities to remain conceptually distinct.

The system is also designed around different categories of users because the responsibilities of a residential society are distributed among different people. An administrator may require broad access to society records, whereas a treasurer primarily deals with financial information. A secretary may work with residents, visitors, complaints, notices, amenities, and bookings, while security personnel require access mainly to visitor-related operations. Residents require access to services relevant to their own participation in society activities. This role-oriented approach is important because a common management system should not assume that every user needs access to every operation. Instead, the interface can present functionality according to the responsibilities associated with a particular role. The project design identifies Resident, Staff, Committee, and Administrator as major user categories within the overall system flow, while the implemented frontend further organizes access according to eight specific operational roles.

Another important aspect of the project is the use of a relational database design. The system contains ten major entities: flats, residents, visitors, staff, maintenance\_bills, payments, complaints, notices, amenities, and amenity\_bookings. These entities represent different areas of society management while maintaining relationships wherever information is naturally connected. Such a design provides a structured foundation for maintaining consistency and reducing unnecessary duplication of information.

The project is therefore intended not merely as a collection of web pages but as a structured software system demonstrating how residential community operations can be modeled using modern web technologies and relational database principles. The frontend provides the visible interaction layer, while the service architecture establishes a foundation for future backend integration. The project design explicitly identifies the backend API as an integration point between the React and Vite application and the MySQL `society_management` database. This architectural decision is important because a production-ready application should not allow a browser-based frontend to directly access the database. Instead, requests should pass through an appropriate server-side API where authentication, authorization, validation, business rules, and database operations can be controlled.

The Society Management Portal therefore represents an attempt to bring several common residential society activities into one organized digital platform. The system focuses on the practical requirements of day-to-day society administration while also demonstrating important concepts in software development, including modular architecture, reusable components, relational database design, role-based access, CRUD operations, state management, routing, user-interface design, and future API integration. Its ten-table relational structure separates property, people, finance, communication, service, and amenity concerns while preserving relationships through foreign keys and exposing those areas through a responsive role-based interface.

## 1.2 Problem Statement

Most small and medium residential societies in India still run their daily operations through a mixture of paper registers, spreadsheets, and messaging groups. The security guard writes visitor entries into a notebook at the gate, the treasurer tracks maintenance dues in a spreadsheet that only one person can edit, complaints arrive as phone calls or WhatsApp messages, and notices are pasted on a board in the lobby or forwarded through chat groups where they are quickly buried under other messages. Each of these practices works in isolation, but together they create a fragmented picture of the society in which no single person can easily answer basic questions such as "Which flats have unpaid bills this month?", "Who is currently inside the premises?", or "How many complaints are still open?"

This fragmentation produces several concrete problems. First, **information is duplicated and inconsistent**: the same resident's phone number may exist in three different places, and when it changes only one copy is updated. Second, **financial tracking is slow and error-prone**: matching payments to bills manually takes time, and residents often have to ask the treasurer to confirm whether their payment was recorded. Third, **accountability is weak**: complaints communicated verbally have no record of when they were raised, who is handling them, or whether they were resolved. Fourth, **gate security depends on handwriting**: visitor registers are difficult to search, are easily lost, and cannot tell a resident in real time that their guest has arrived. Fifth, **shared amenities are double-booked** because there is no single schedule that everyone can see.

A further problem is that existing commercial society-management applications are often designed for large gated communities, are subscription-based, and expose every feature to every user. Small committees find them expensive and complicated, while residents and staff are confronted with screens that are irrelevant to them. There is a need for a lightweight, role-aware system in which each person—administrator, committee member, treasurer, secretary, resident, or staff member—sees only the information and actions that match their responsibility.

The problem addressed by this project can therefore be stated as follows: *to design and develop a centralized, responsive, role-based web portal that digitizes the core operations of a residential society—flats, residents, visitors, staff, maintenance billing, payments, complaints, notices, amenities, and amenity bookings—on top of a normalized relational data model, so that records are consistent, searchable, and accessible to the right people at the right time.*

### 1.3 Scope of the Project

The scope of the project defines what the Society Management Portal covers in its current release and what is intentionally left for later phases. The current release is a complete, working frontend application built with React 19 and Vite 8, together with a fully defined MySQL data model and a service layer that is ready to be connected to a backend API.

#### *In scope (implemented)*

- **Ten functional modules** mapped one-to-one to the database tables: Flats, Residents, Visitors, Staff, Maintenance Bills, Payments, Complaints, Notices, Amenities, and Amenity Bookings.
- **Create, read, update, and delete (CRUD)** operations for each module through reusable forms, tables, and confirmation dialogs.
- **Eight user roles**—Admin, Committee, Treasurer, Secretary, Resident, Security, Housekeeping, and Maintenance Staff—each with its own dashboard, navigation menu, and permitted actions.
- **Record-level scoping**, so that a resident sees only the bills, payments, visitors, complaints, and bookings that belong to their own flat or account.
- **Admin analytics dashboard** with summary cards and charts for occupancy, maintenance status, and complaint progress, built with Recharts.
- **Search, filter, sort, and pagination** on every data table, plus a global search bar in the top navigation.
- **Client-side validation** for required fields, email format, and phone number format.
- **Responsive layout** that works on desktops, tablets, and mobile phones, with a collapsible sidebar on small screens.
- **Demo authentication** with eight pre-configured accounts and a role selector on the login page.

#### *Out of scope for the current release (planned)*

- A production backend (Node.js and Express REST API) with server-side authentication, password hashing, and authorization.
- Persistent storage in the MySQL society\_management database; the current release uses an in-memory store seeded with realistic sample data.

- Real online payment processing through a payment gateway; payments are currently recorded as transactions with a method and transaction ID.
- Email, SMS, and push notifications; the notification panel currently displays sample alerts.
- Native Android and iOS applications.

Keeping these items outside the current scope allowed the team to focus on getting the data model, the user interface, and the role-based access rules right first. Because all data access passes through a single service file, the planned backend can be added later without rewriting the user interface.

## 1.4 Objectives

The project was guided by the following objectives:

1. **Centralize society records.** Bring flats, residents, visitors, staff, bills, payments, complaints, notices, amenities, and bookings into one application backed by a single relational schema.
2. **Enforce role-based access.** Ensure that each of the eight roles sees only the pages, records, and actions appropriate to its responsibilities, with all permission rules defined in one configuration file.
3. **Build reusable, configuration-driven components.** Develop a generic resource page, data table, and record form so that new modules can be added by writing configuration rather than new code.
4. **Provide useful dashboards.** Give administrators an analytical overview of occupancy, dues, complaints, and visitors, and give every other role a focused dashboard with quick actions.
5. **Ensure data quality at entry.** Validate required fields and formats before records are saved, and use dropdowns for foreign-key relationships so that users choose existing flats, residents, bills, and amenities instead of typing IDs.
6. **Deliver a responsive, accessible interface.** Make every screen usable on mobile phones as well as desktops, using clear typography, status badges, and consistent layout.
7. **Prepare for backend integration.** Isolate all data operations in an asynchronous service layer so that the in-memory store can later be replaced by HTTP calls to a REST API without changes to the pages.
8. **Apply sound engineering practice.** Use version control, linting, a production build pipeline, and structured testing throughout development.

## 1.5 Importance of the Application

A society management portal matters because it directly affects the daily experience of every person living or working in a residential community. For the **managing committee**, it replaces scattered

registers with one reliable source of truth. Decisions about maintenance, staffing, or amenity rules can be based on current numbers—how many flats are occupied, how much money is pending, how many complaints remain open—rather than on estimates.

For the **treasurer**, the application turns the monthly collection cycle into a transparent process. Every bill is linked to a flat, every payment is linked to both a bill and a resident, and every transaction carries a method and a transaction ID. Pending and overdue amounts are calculated automatically on the dashboard, which reduces follow-up effort and the chance of disputes.

For **residents**, the portal offers self-service. A resident can see their own dues and payment history, raise a complaint and follow its status, pre-register a visitor, and book an amenity without calling the office. Because records are scoped to their own flat, residents do not see other families' information, which protects privacy.

For **security and service staff**, the portal provides simple, focused tools. The security guard sees a visitor dashboard showing who is inside, who is expected, and who has exited. Maintenance staff see the complaint queue and can update progress. Housekeeping staff receive notices without having to be added to resident chat groups.

From an **academic and technical** perspective, the project is important because it demonstrates how a real-world domain can be modelled as a normalized relational schema and exposed through a modern component-based frontend. It applies concepts from database design, software architecture, user-interface design, access control, and testing in a single coherent system.

## 1.6 Target Users / Audience

The portal is designed for the following user groups. Each group corresponds to one or more roles defined in `src/config/permissions.js`.

| User group                 | Role(s) in the system | Primary needs                                                                                   |
|----------------------------|-----------------------|-------------------------------------------------------------------------------------------------|
| Society administrator      | Admin                 | Full control of all ten modules, analytics dashboard, user and record management.               |
| Managing committee         | Committee             | Operational overview; manage residents, visitors, complaints, notices, amenities, and bookings. |
| Finance officer            | Treasurer             | Create and track maintenance bills, record and review payments, view residents and flats.       |
| Communications and records | Secretary             | Publish notices, manage complaints, visitors, amenities, and bookings.                          |
| Flat owners and tenants    | Resident              | View own dues and payments, raise complaints, register visitors, book amenities, read notices.  |

|                         |                   |                                                                          |
|-------------------------|-------------------|--------------------------------------------------------------------------|
| Gate security           | Security          | Register and update visitor entries and exits; read notices.             |
| Housekeeping staff      | Housekeeping      | Staff dashboard with announcements and service coverage information.     |
| Maintenance technicians | Maintenance Staff | View the complaint queue and update complaint status as work progresses. |

The secondary audience includes developers and students who wish to study or extend the system. The configuration-driven design and the clear separation between interface, state, and service layers make the codebase a practical reference for building similar management applications for hostels, office complexes, or gated communities.

## CHAPTER -2 SYSTEM REQUIREMENTS

### 2.1 Hardware Requirements

Because the Society Management Portal is a browser-based single-page application, its hardware requirements are modest. The requirements are listed separately for the development environment, for end users, and for the planned server environment.

#### *Development machine*

| Component    | Minimum                                     | Recommended                                                       |
|--------------|---------------------------------------------|-------------------------------------------------------------------|
| Processor    | Dual-core 2.0 GHz (Intel i3 / Ryzen 3)      | Quad-core 2.5 GHz or better (Intel i5 / Ryzen 5 / Apple M-series) |
| Memory (RAM) | 4 GB                                        | 8 GB or more (Node.js, browser, and editor running together)      |
| Storage      | 2 GB free (project plus node_modules)       | SSD with 10 GB free                                               |
| Display      | 1366 × 768                                  | 1920 × 1080, for side-by-side code and browser                    |
| Network      | Internet access for installing npm packages | Broadband connection for Git and deployment                       |

#### *End-user devices*

| Device type          | Requirement                                                                                             |
|----------------------|---------------------------------------------------------------------------------------------------------|
| Desktop / laptop     | Any computer able to run a current version of Chrome, Edge, Firefox, or Safari; 2 GB RAM or more.       |
| Tablet               | Android or iPadOS tablet with a modern browser; the layout adapts to medium screens.                    |
| Mobile phone         | Android 9+ or iOS 15+ phone with a screen width of 360 px or more; the sidebar becomes a slide-in menu. |
| Gate desk (security) | Low-cost tablet or laptop at the gate with Wi-Fi or mobile data.                                        |

#### *Server (planned production environment)*

| Component      | Requirement                                                                                                                                            |
|----------------|--------------------------------------------------------------------------------------------------------------------------------------------------------|
| Static hosting | Any static host or CDN (Vercel, Netlify, GitHub Pages, Nginx). The production build is about 0.7 MB of JavaScript and 26 KB of CSS before compression. |
| API server     | 1 vCPU, 1–2 GB RAM virtual machine or platform service for the Node.js/Express API.                                                                    |

|                 |                                                                                                             |
|-----------------|-------------------------------------------------------------------------------------------------------------|
| Database server | MySQL 8.0 instance with 1 vCPU, 1 GB RAM, and 10 GB storage, which is sufficient for several hundred flats. |
|-----------------|-------------------------------------------------------------------------------------------------------------|

## 2.2 Software Requirements

| Category          | Software                                                                     | Purpose                                                    |
|-------------------|------------------------------------------------------------------------------|------------------------------------------------------------|
| Operating system  | Windows 10/11, macOS 12+, or Ubuntu 22.04+                                   | Development and testing                                    |
| Runtime           | Node.js 20 LTS or later (with npm 10)                                        | Running Vite, installing packages, building                |
| Frontend library  | React 19.2 and React DOM 19.2                                                | Component-based user interface                             |
| Build tool        | Vite 8.2 with @vitejs/plugin-react 6.1                                       | Development server with hot reload and production bundling |
| Routing           | React Router DOM 7.18                                                        | Client-side navigation and protected routes                |
| Charts            | Recharts 3.10                                                                | Pie and bar charts on the admin dashboard                  |
| Icons             | Lucide React 1.42                                                            | Consistent SVG icon set                                    |
| Code quality      | ESLint 10 with react-hooks and react-refresh plugins                         | Static analysis and hook-rule checks                       |
| Database (target) | MySQL 8.0 (database society_management)                                      | Persistent relational storage                              |
| Backend (planned) | Node.js with Express, mysql2, bcrypt, jsonwebtoken                           | REST API, authentication, authorization                    |
| Browser           | Google Chrome, Microsoft Edge, Mozilla Firefox, Safari (latest two versions) | Running the application                                    |
| Version control   | Git 2.40+ and a GitHub account                                               | Source control and collaboration                           |

## 2.3 Development Tools and Frameworks

**Visual Studio Code** was the primary code editor. Its integrated terminal was used to run the Vite development server, and extensions such as ESLint, Prettier, and ES7 React snippets helped keep the JSX code consistent. The built-in Git panel was used for staging and committing changes.

**Vite** served as both the development server and the production bundler. During development, Vite serves source files as native ES modules and applies hot module replacement, so edits to a component appear in the browser almost instantly without losing state. For production, npm run build bundles and minifies the application into the dist/ folder.

**React** is the core framework. The application is composed of function components that use hooks—`useState`, `useEffect`, `useMemo`, and `useContext`—for state, side effects, memoization, and shared data. React's `StrictMode` is enabled in `main.jsx` to surface unsafe patterns during development. **React Router** provides declarative routing. All protected pages are nested under a single layout route wrapped in a `ProtectedRoute` component, which checks the signed-in user and the role permissions before rendering.

**Recharts** is used for the admin dashboard visualizations: a donut chart for flat occupancy, a vertical bar chart for maintenance bill status, and a horizontal bar chart for complaint status. Recharts was chosen because it is built specifically for React and its charts are responsive by default.

**Lucide React** supplies the icon set used in the sidebar, stat cards, buttons, and quick actions. Each icon is imported individually, which keeps the bundle smaller than importing an entire icon font.

**ESLint** enforces code-quality rules. The project uses the recommended JavaScript rules plus `eslint-plugin-react-hooks`, which catches incorrect hook usage, and `eslint-plugin-react-refresh`, which ensures components are compatible with fast refresh. The final codebase passes `npm run lint` with zero errors and zero warnings.

**Git and GitHub** are used for version control. The repository ignores generated folders such as `node_modules` and `dist` through `.gitignore`, and the recommended workflow is to develop each feature on its own branch and merge it into the main branch after review.

**Browser developer tools** (Chrome DevTools) were used to inspect the component tree, debug layout issues with the responsive grid, and monitor console warnings. The device-emulation mode was used to test the mobile layout.

**Playwright**, a browser-automation framework, was used to run end-to-end test scenarios against the running application across different roles and to capture the screenshots included in the appendices.

**Postman** is the tool selected for testing the REST API of the planned backend. The API contract—each endpoint, request body, and expected response—has been derived from the service layer (see Section 8.1) so that the backend and the frontend can be developed against the same specification.

**Graphviz** and diagramming tools were used to prepare the architecture diagram and the entity-relationship diagram.

## CHAPTER -3 TECHNOLOGY STACK

The technology stack of the Society Management Portal was selected to keep the application fast to develop, easy to maintain, and straightforward to connect to a relational database later. The table below summarizes the stack before each layer is discussed in detail.

| Layer              | Technology                                          | Status in current release                        |
|--------------------|-----------------------------------------------------|--------------------------------------------------|
| Presentation       | React 19, JSX, HTML5, CSS3                          | Implemented                                      |
| Build & dev server | Vite 8                                              | Implemented                                      |
| Routing            | React Router DOM 7                                  | Implemented                                      |
| State management   | React Context API + hooks                           | Implemented                                      |
| Charts & icons     | Recharts 3, Lucide React                            | Implemented                                      |
| Service layer      | Asynchronous JavaScript module<br>(api.js, auth.js) | Implemented with in-memory data                  |
| Backend            | Node.js + Express REST API                          | Designed, planned for next phase                 |
| Database           | MySQL 8 ( society_management)                       | Schema designed; sample data mirrors all columns |
| Version control    | Git, GitHub                                         | In use                                           |

### 3.1 Front-End (React.js, HTML/CSS)

The front end is a single-page application (SPA) written in React 19. In a single-page application the browser loads one HTML document (index.html) and a JavaScript bundle once; afterwards, React renders and replaces screens in the browser without full page reloads. This gives the portal the responsiveness of a desktop application: moving from the Residents page to the Payments page happens instantly, and the shared sidebar and navigation bar remain on screen.

#### ***Component-based structure***

React encourages building the interface from small, reusable components. The project contains three groups of components:

- **Layout components** – DashboardLayout, Sidebar, and Navbar define the application shell: a dark navigation sidebar grouped into Overview, Society, Gate, Finance, Community, and Account sections, and a top bar with global search, notifications, and a profile menu.
- **Shared UI components** – DataTable, RecordForm, Modal, StatCard, StatusBadge, Toast, EmptyState, and RoleBadge are used across many pages.

- **Page components** – Dashboard, RoleDashboard, ResourcePage, Notices, Amenities, Settings, and Login. Seven modules (Flats, Residents, Visitors, Staff, Maintenance, Payments, Complaints, Bookings) are thin wrappers that render ResourcePage with a different resource name.

### ***Configuration-driven pages***

A key design decision was to describe each module declaratively in `src/config/resources.js`. For every resource, the configuration lists the page title, the primary-key field, the table columns and how each should be displayed (plain text, status badge, currency, date, or a relationship such as flat or resident), the form fields with their types and validation rules, the filter fields, and the summary statistics. The generic ResourcePage reads this configuration and produces a complete page with statistics cards, a search box, filters, a sortable and paginated table, and add, edit, view, and delete dialogs. As a result, the Flats, Residents, Visitors, Staff, Maintenance, Payments, Complaints, and Bookings pages each require only a two-line page file.

### ***HTML5 and CSS3***

Semantic HTML5 elements such as header, main, aside, nav, section, article, and footer give the pages a meaningful structure. Native form controls are used with the appropriate input types— email, tel, number, date, time, and datetime-local—so that mobile browsers show suitable keyboards and pickers. Dialogs carry `role="dialog"` and `aria-modal` attributes, and icon-only buttons carry `aria-label` text.

Styling is written in plain CSS (`index.css`) without a CSS framework. CSS Grid lays out the dashboard cards and charts, and Flexbox aligns toolbars, table actions, and list items. Media queries adapt the layout for tablets and phones: the summary grid collapses from six columns to two or one, and the sidebar becomes an off-canvas menu opened from a hamburger button. A consistent colour system is used for status badges—green for Paid, Successful, Resolved, and Exited; amber for Pending and Expected; red for Overdue, Failed, and Open; and blue for Inside and Confirmed—so that users can understand status at a glance.

### ***Vite***

Vite was chosen over older tools such as Create React App because it starts the development server in well under a second and updates modules instantly through hot module replacement. For production, Vite (version 8, using the Rolldown bundler) produces optimized, hashed asset files in the dist directory. The current production build contains 2,458 transformed modules and outputs a 686 KB JavaScript bundle (202 KB gzipped) and a 26 KB stylesheet (6 KB gzipped).

### 3.2 Back-End (Node.js / Express)

The back end is the layer that will receive requests from the React application, apply business rules and security checks, and read from or write to the MySQL database. In the current release, the back end is represented by the service layer (`src/services/api.js`), which exposes asynchronous `list`, `add`, `update`, and `remove` functions for every resource and simulates network latency of 180 milliseconds. This allowed the complete user interface to be built and tested before the server was written.

For the production back end, **Node.js with the Express framework** has been selected. The reasons for this choice are:

- **One language across the stack.** The team writes JavaScript on both the client and the server, which reduces context switching and allows validation logic to be shared.
- **Non-blocking I/O.** Node.js handles many simultaneous requests efficiently, which suits a portal where many residents may check dues at the start of the month.
- **Mature ecosystem.** Well-tested packages exist for every requirement: `mysql2` for database access with prepared statements, `bcrypt` for password hashing, `jsonwebtoken` for stateless authentication, `express-validator` for request validation, `helmet` for security headers, and `cors` for cross-origin configuration.
- **Simple REST design.** Express routers map naturally to the ten resources, for example `GET /api/v1/flats` or `PUT /api/v1/complaints/:id`.

The planned back end follows a layered structure: **routes** define the URL endpoints; **middleware** verifies the JSON Web Token, checks the user's role against the same permission rules used in the frontend, and validates the request body; **controllers** coordinate the request; and **repositories** execute parameterized SQL statements. Every permission that is currently enforced in the browser will be enforced again on the server, because hiding a button in the interface is a usability measure, not a security control.

### 3.3 Database (MySQL)

MySQL 8 was chosen as the database because society data is naturally relational. Flats have residents; flats receive bills; bills receive payments made by residents; residents raise complaints and book amenities. These relationships are best represented with tables, primary keys, and foreign keys, and they benefit from the integrity guarantees of a relational database management system.

The database, named `society_management`, contains ten tables: `flats`, `residents`, `visitors`,

`staff`, `maintenance_bills`, `payments`, `complaints`, `notices`, `amenities`, and `amenity_bookings`.

The schema is normalized so that each fact is stored once—for example, a resident's flat is stored as

a `flat_id` foreign key rather than repeating the block and flat number. Enumerated columns (ENUM) restrict status fields to valid values such as *Paid / Pending / Overdue* or *Open / In Progress / Resolved*, and these same values are used for the dropdowns in the frontend forms.

Key advantages of MySQL for this project are:

- **Referential integrity** through foreign-key constraints, which prevents, for example, a payment being recorded against a bill that does not exist.
- **ACID transactions**, which will be used when a payment is recorded and the corresponding bill status must be updated in the same atomic operation.
- **Indexing** on foreign keys and frequently filtered columns such as `bill_status`, `complaint_status`, and `booking_date`.
- **Wide hosting support** on every major cloud provider and on low-cost shared hosting, which is important for small societies.

The sample data in `src/data/mockData.js` mirrors the exact column names and foreign-key relationships of the MySQL tables: 12 flats across three blocks, 10 residents, 8 visitors, 6 staff members, 11 maintenance bills, 8 payments, 6 complaints, 5 notices, 5 amenities, and 8 amenity bookings. Because the frontend already uses the database column names, connecting to the real database will not require changes to the table or form configuration.

### 3.4 Version Control (Git, GitHub)

Git is used to track every change to the source code, and GitHub hosts the remote repository. Version control serves three purposes in this project. First, it keeps a complete history of the code, so that any change can be reviewed or reverted. Second, it allows the three team members to work on different parts of the application in parallel—such as the dashboards, the resource configuration and forms, or authentication and permissions—and then merge their work. Third, it provides the starting point for automated checks and deployment.

The repository includes a `.gitignore` file that excludes `node_modules`, the dist build output, log files, and editor settings, so that only source files are committed. The `package-lock.json` file is committed so that every developer and every build server installs exactly the same package versions. The recommended workflow is:

1. Create a feature branch from main (for example `feature/complaints-module`).
2. Commit small, focused changes with descriptive messages.
3. Run `npm run lint` and `npm run build` locally before pushing.
4. Open a pull request on GitHub for review by another team member.

5. Merge into `main`; the hosting platform then builds and deploys the new version automatically (see Chapter 9).

### 3.5 APIs and External Services

The application deliberately keeps external dependencies to a minimum so that it can run offline for demonstrations and remain inexpensive for small societies. The libraries and interfaces it relies on are:

| API / service                                      | Use in the project                                                                                                                                                                                                                                                                           |
|----------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Internal service API<br>( <code>api.js</code> )    | Single access point for all data. Provides list, add, update, and remove, plus named helpers such as <code>getFlats</code> , <code>addResident</code> , <code>updateComplaint</code> , <code>deleteBooking</code> , <code>getMaintenanceBills</code> , and <code>getAmenityBookings</code> . |
| Authentication service<br>( <code>auth.js</code> ) | <code>signIn</code> , <code>signOut</code> , and <code>getCurrentUser</code> ; stores the signed-in user (without the password) under the <code>haven-demo-user</code> key in <code>localStorage</code> .                                                                                    |
| Web Storage API                                    | Browser <code>localStorage</code> keeps the user signed in across page reloads.                                                                                                                                                                                                              |
| Intl API                                           | <code>Intl.NumberFormat('en-IN', {currency: 'INR'})</code> formats amounts in Indian Rupees (for example ₹36,000), and <code>Intl.DateTimeFormat('en-IN')</code> formats dates such as 08 Sept 2026.                                                                                         |
| Recharts API                                       | Declarative chart components ( <code>PieChart</code> , <code>BarChart</code> , <code>ResponsiveContainer</code> , <code>Tooltip</code> ).                                                                                                                                                    |
| Lucide icon API                                    | Tree-shakable SVG icons imported as React components.                                                                                                                                                                                                                                        |
| REST API (planned)                                 | Express endpoints under <code>/api/v1</code> for all ten resources plus <code>/auth/login</code> ; documented in Section 8.1.                                                                                                                                                                |
| Payment gateway<br>(planned)                       | Razorpay or a similar UPI-enabled gateway for online maintenance payments; the payment table already stores method, transaction ID, and status.                                                                                                                                              |
| Notification service<br>(planned)                  | Email (for example Nodemailer with SMTP) or SMS alerts for new bills, visitor arrivals, and complaint updates.                                                                                                                                                                               |
| Notification service<br>(planned)                  |                                                                                                                                                                                                                                                                                              |

## CHAPTER -4 SYSTEM ARCHITECTURE

### 4.1 High-Level Architecture Diagram

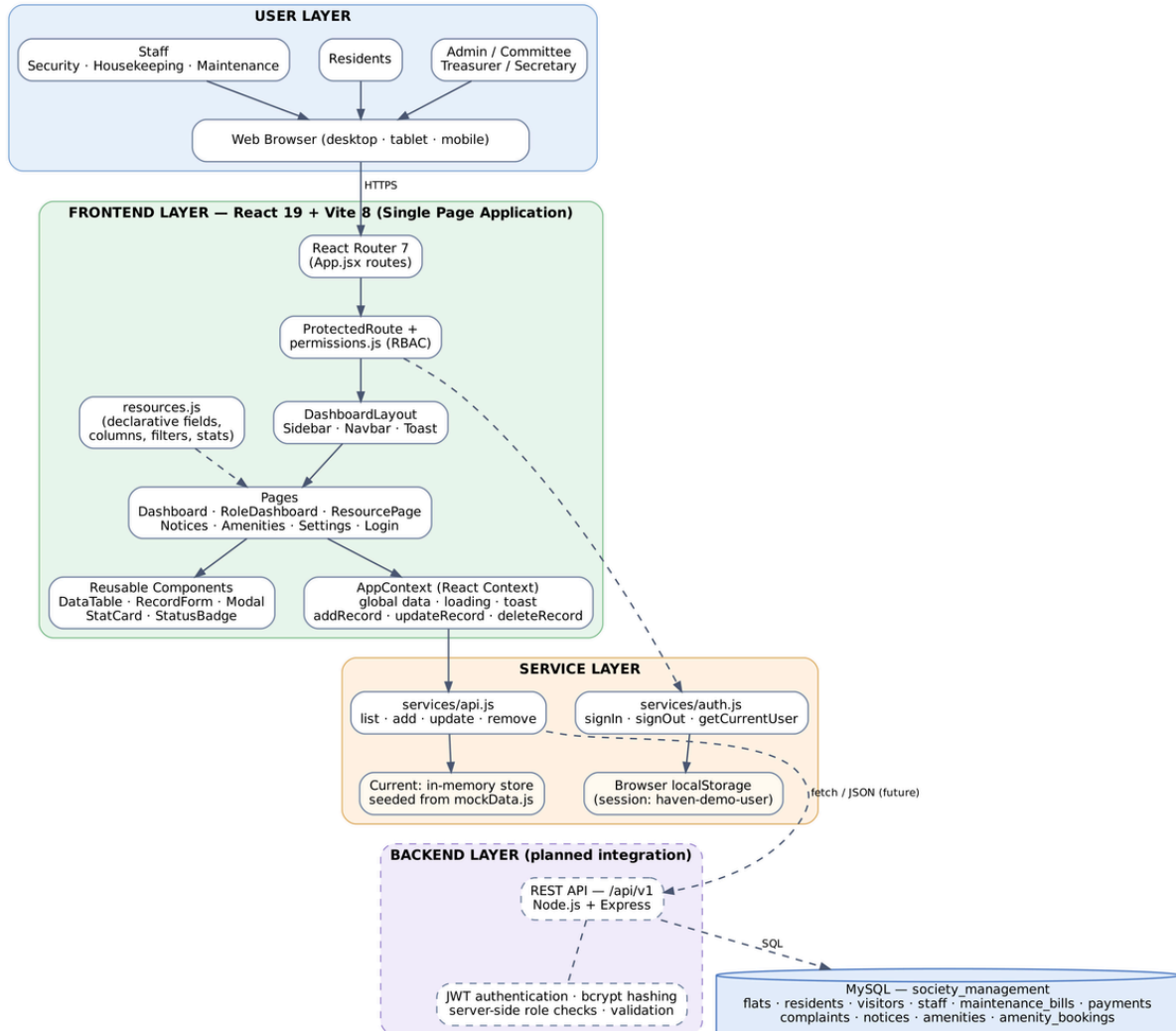


Figure 4.1 – High-level architecture of the Society Management Portal (solid lines: implemented; dashed lines: planned backend integration)

The architecture follows a layered approach. Users in eight roles access the portal from a web browser. The React single-page application handles routing, permission checks, layout, pages, and shared components. Pages read and change data only through a shared React Context, which in turn calls the service layer. In the current release, the service layer works against an in-memory store seeded from realistic sample data; in the next phase, the same functions will send HTTP requests to an Express REST API, which will apply authentication and authorization and run SQL queries against the MySQL society\_management database.

## 4.2 Description of Each Layer (Frontend, Backend, Database)

### *User layer*

The user layer consists of the people who use the portal and the devices they use. Users are divided into three broad groups: management users (Admin, Committee, Treasurer, Secretary), residents, and staff (Security, Housekeeping, Maintenance Staff). All of them use an ordinary web browser; no installation is required. Because the layout is responsive, the same application serves a treasurer on a laptop, a resident on a phone, and a security guard on a tablet at the gate.

### *Frontend layer*

The frontend layer is the React application. Its responsibilities are divided among several clearly separated parts:

- **Routing (App.jsx).** React Router defines a public `/login` route and a protected layout route containing the dashboard and the eleven module pages (`/flats`, `/residents`, `/visitors`, `/staff`, `/maintenance`, `/payments`, `/complaints`, `/notices`, `/amenities`, `/bookings`, `/settings`). Any unknown URL redirects to the dashboard.
- **Access control (ProtectedRoute and permissions.js).** Before any protected page renders, ProtectedRoute confirms that a user is signed in and that the user's role is allowed to open the requested path. If not, the user is redirected to the login page or to the first page their role may access.
- **Layout (DashboardLayout).** Renders the sidebar (filtered by role), the top navigation bar, the routed page through React Router's Outlet, and a toast for success messages.
- **Pages and components.** The generic ResourcePage turns the configuration in resources.js into a complete management page, while specialised pages handle the dashboards, notices, amenities, settings, and login.
- **State (AppContext).** A single context provider loads all ten resources when the application starts and exposes data, loading, toast, and three mutation functions: `addRecord`, `updateRecord`, and `deleteRecord`. Every page reads from and writes through this context, so that a change made on one page—for example resolving a complaint—is immediately reflected on the dashboard.

### *Service layer*

The service layer separates the user interface from the data source. `api.js` exposes asynchronous CRUD functions and generates convenient named helpers for each resource. `auth.js` handles sign-in, sign-out, and reading the current user. Because every function returns a Promise, the pages already handle data asynchronously, exactly as they will when real network calls are introduced.

This is the single replacement point for backend integration: changing the body of list, add, update, and remove to call `fetch()` against the REST API is sufficient to switch the whole application from sample data to the live database.

### ***Backend layer (planned)***

The backend will be a Node.js and Express REST API. It will authenticate users against the residents table (and a staff/admin accounts table) using bcrypt-hashed passwords, issue a JSON Web Token on login, and verify that token on every subsequent request. Middleware will re-apply the same role rules that the frontend uses, validate request bodies, and return consistent JSON error messages. Business rules that span tables—such as marking a bill as Paid when a successful payment is recorded, or rejecting an amenity booking that overlaps an existing one—belong in this layer.

### ***Database layer***

The database layer is a MySQL 8 database named `society_management` with ten normalized tables connected by foreign keys (see Chapter 5). The database enforces data integrity independently of the application: primary keys guarantee uniqueness, foreign keys guarantee valid relationships, ENUM columns restrict statuses to valid values, and unique constraints prevent duplicate emails and transaction IDs.

### ***Data flow example***

The following sequence shows how a typical operation—a resident raising a complaint—passes through the layers:

1. The resident clicks *Raise complaint* on their dashboard; React Router navigates to `/complaints`, and `ProtectedRoute` confirms that the Resident role may open it.
2. `ResourcePage` shows only the resident's own complaints (filtered by `scopeRecords`) and an *Add Complaint* button (allowed by `canCreateResource`).
3. The resident fills in the title, description, date, and status. The *Resident* field is hidden for residents; the page attaches the resident's own `resident_id` automatically.
4. `RecordForm` validates the fields; `ResourcePage` converts ID fields to numbers and calls `addRecord('complaints', values)` from the context.
5. The context calls `api.add`, which assigns the next `complaint_id` and returns the new record (in future: `POST /api/v1/complaints → INSERT INTO complaints ...`).
6. The context updates its state and shows the toast "Complaint added successfully."; the table, the resident dashboard, and the admin complaint chart all update automatically.

### 4.3 Deployment Architecture (Cloud, Local, CI/CD Pipelines)

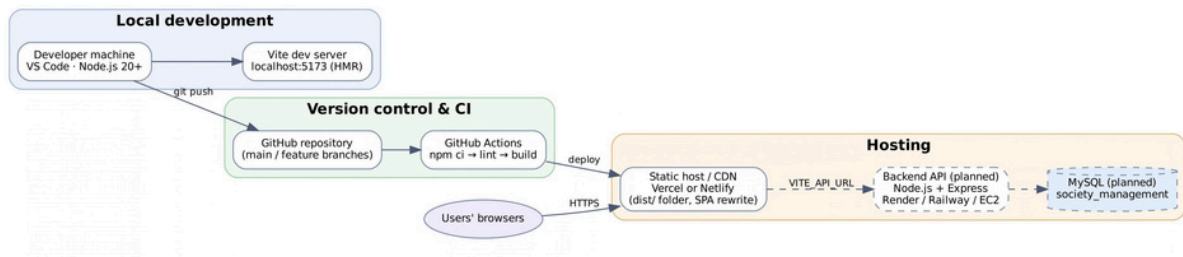


Figure 4.2 – Deployment architecture (dashed: planned backend and database hosting)

**Local environment.** During development, each team member runs the Vite development server on `http://localhost:5173`. Vite serves the source files directly with hot module replacement, so changes appear in the browser as soon as a file is saved. Because the current service layer uses in- memory sample data, the full application—including all eight roles—runs locally without a database or server.

**Version control and continuous integration.** Source code is pushed to a GitHub repository. A continuous-integration workflow (for example GitHub Actions) can run on every push and pull request with three steps: `npm ci` to install the exact locked dependency versions, `npm run lint` to check code quality, and `npm run build` to confirm that the production bundle compiles. A pull request is merged only when these checks pass.

**Frontend hosting.** The output of `npm run build` is a static `dist/` folder containing `index.html`, one hashed JavaScript file, and one hashed CSS file. This folder can be served by any static host or content delivery network. Vercel and Netlify are the recommended platforms because they connect directly to the GitHub repository, build automatically on every push to main, provide free HTTPS certificates, and create preview deployments for pull requests. Because the application uses client-side routing, the host is configured to rewrite all paths to `index.html`, so that refreshing a page such as `/payments` does not return a 404 error.

**Backend and database hosting (planned).** The Express API will be deployed to a Node.js hosting service such as Render, Railway, or an AWS EC2 instance, and the MySQL database to a managed service such as AWS RDS, Railway MySQL, or Aiven. The frontend will read the API base URL from an environment variable (`VITE_API_URL`), and secrets such as the database password and JWT signing key will be stored as environment variables on the server, never in the repository.

**Security in deployment.** All traffic is served over HTTPS. The API will enable CORS only for the portal's own domain, set security headers with Helmet, use parameterized queries to prevent SQL injection, and apply rate limiting to the login endpoint. Regular automated database backups will protect society records against accidental loss.

## CHAPTER -5 DESIGN

### 5.1 ER Diagram



Figure 5.1 – Entity-relationship diagram of the society\_management database

The diagram shows ten entities. **FLATS** is the central property entity: one flat can have many residents, many visitors, and many maintenance bills. **RESIDENTS** is the central people entity: one resident can make many payments, raise many complaints, and make many amenity bookings. Each **PAYMENT** belongs to exactly one maintenance bill and one resident, and each **AMENITY\_BOOKING** links one resident to one amenity. **STAFF** and **NOTICES** are independent entities that do not reference other tables.

## 5.2 Database Schema

The relationships in the ER diagram are summarized below, followed by the column definitions of each table.

| Parent table      | Child table       | Foreign key | Cardinality | Meaning                                        |
|-------------------|-------------------|-------------|-------------|------------------------------------------------|
| flats             | residents         | flat_id     | 1 : N       | A flat houses one or more residents            |
| flats             | visitors          | flat_id     | 1 : N       | A visitor comes to a specific flat             |
| flats             | maintenance_bills | flat_id     | 1 : N       | Each month's bill is raised per flat           |
| maintenance_bills | payments          | bill_id     | 1 : N       | A bill may be paid in one or more transactions |
| residents         | payments          | resident_id | 1 : N       | The resident who made the payment              |
| residents         | complaints        | resident_id | 1 : N       | The resident who raised the complaint          |
| residents         | amenity_bookings  | resident_id | 1 : N       | The resident who booked the amenity            |
| amenities         | amenity_bookings  | amenity_id  | 1 : N       | The amenity that was booked                    |

| flats            |                                   |                                                |
|------------------|-----------------------------------|------------------------------------------------|
| flat_id          | INT, PK, AUTO_INCREMENT           | Unique flat identifier                         |
| flat_number      | VARCHAR(10), NOT NULL             | Number of the flat within its block (e.g. 101) |
| block_name       | VARCHAR(10), NOT NULL             | Block or tower (A, B, C)                       |
| floor_number     | INT, NOT NULL                     | Floor on which the flat is located             |
| flat_type        | ENUM('1BHK','2BHK','3BHK','4BHK') | Size category                                  |
| occupancy_status | ENUM('Occupied','Vacant')         | Current occupancy                              |

| residents     |                         |                                      |
|---------------|-------------------------|--------------------------------------|
| resident_id   | INT, PK, AUTO_INCREMENT | Unique resident identifier           |
| resident_name | VARCHAR(100), NOT NULL  | Full name                            |
| email         | VARCHAR(100), UNIQUE    | Login email                          |
| phone         | VARCHAR(15), NOT NULL   | 10–15 digit mobile number            |
| password      | VARCHAR(255), NOT NULL  | Password hash (bcrypt in production) |

|               |                                                      |                            |
|---------------|------------------------------------------------------|----------------------------|
| role          | ENUM('Resident','Committee','Treasurer','Secretary') | Portal role                |
| resident_type | ENUM('Owner','Tenant')                               | Ownership status           |
| flat_id       | INT, FK → flats                                      | Flat the resident lives in |

| visitors                     |                                    |                                                        |
|------------------------------|------------------------------------|--------------------------------------------------------|
| visitor_id                   | INT, PK, AUTO_INCREMENT            | Unique visitor entry                                   |
| visitor_name, phone, purpose | VARCHAR                            | Who is visiting and why (Delivery, Guest, Plumber ...) |
| flat_id                      | INT, FK → flats                    | Destination flat                                       |
| entry_time / exit_time       | DATETIME / DATETIME NULL           | Gate timestamps; exit is empty while inside            |
| status                       | ENUM('Inside','Exited','Expected') | Current visit status                                   |

| staff             |                                                          |                         |
|-------------------|----------------------------------------------------------|-------------------------|
| staff_id          | INT, PK, AUTO_INCREMENT                                  | Unique staff identifier |
| staff_name, phone | VARCHAR                                                  | Name and contact number |
| staff_type        | ENUM('Security','Housekeeping','Maintenance','Gardener') | Department              |
| address           | TEXT                                                     | Residential address     |

| maintenance_bills |                                  |                                      |
|-------------------|----------------------------------|--------------------------------------|
| bill_id           | INT, PK, AUTO_INCREMENT          | Unique bill number                   |
| flat_id           | INT, FK → flats                  | Flat being billed                    |
| bill_month        | VARCHAR(20)                      | Billing period (e.g. September 2026) |
| amount            | DECIMAL(10,2)                    | Amount due in INR                    |
| due_date          | DATE                             | Last date for payment                |
| bill_status       | ENUM('Paid','Pending','Overdue') | Payment status                       |

| payments              |                                         |                                     |
|-----------------------|-----------------------------------------|-------------------------------------|
| payment_id            | INT, PK, AUTO_INCREMENT                 | Unique payment identifier           |
| bill_id / resident_id | INT, FK → maintenance_bills / residents | Bill being paid and resident paying |
| payment_date          | DATE                                    | Date of payment                     |

|                |                                           |                      |
|----------------|-------------------------------------------|----------------------|
| amount         | DECIMAL(10,2)                             | Amount paid          |
| payment_method | ENUM('UPI','Card','Net Banking','Cheque') | Mode of payment      |
| transaction_id | VARCHAR(50), UNIQUE                       | Bank / UPI reference |
| payment_status | ENUM('Successful','Pending','Failed')     | Transaction result   |

| complaints                              |                                       |                     |
|-----------------------------------------|---------------------------------------|---------------------|
| complaint_id                            | INT, PK, AUTO_INCREMENT               | Ticket number       |
| resident_id                             | INT, FK → residents                   | Who raised it       |
| complaint_title / complaint_description | VARCHAR(150) / TEXT                   | Summary and details |
| complaint_date                          | DATE                                  | Date raised         |
| complaint_status                        | ENUM('Open','In Progress','Resolved') | Resolution progress |

| notices · amenities · amenity_bookings |                                                                                                                                                                     |                                                   |
|----------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------|
| notices                                | notice_id PK, notice_title, notice_description, notice_date                                                                                                         | Society announcements                             |
| amenities                              | amenity_id PK, amenity_name, location, availability_status ENUM('Available','Maintenance')                                                                          | Shared facilities such as the pool, gym, and hall |
| amenity_bookings                       | booking_id PK, resident_id FK, amenity_id FK, booking_date DATE, start_time TIME, end_time TIME, booking_status ENUM('Confirmed','Pending','Completed','Cancelled') | Reservations of amenities by residents            |

A representative excerpt of the SQL definition is shown below.

```

CREATE DATABASE society_management;
USE society_management;

CREATE TABLE flats (
  flat_id          INT AUTO_INCREMENT PRIMARY KEY,
  flat_number      VARCHAR(10) NOT NULL,
  block_name       VARCHAR(10) NOT NULL,
  floor_number     INT NOT NULL, ENUM('1BHK','2BHK','3BHK','4BHK')
  flat_type        NOT NULL,
  occupancy_status ENUM('Occupied','Vacant') NOT NULL DEFAULT 'Vacant',
  UNIQUE (block_name, flat_number)
);

CREATE TABLE maintenance_bills (
  bill_id         INT AUTO_INCREMENT PRIMARY KEY,
  flat_id         INT NOT NULL,
  bill_month      VARCHAR(20) NOT NULL,
  amount          DECIMAL(10,2) NOT NULL,
  due_date        DATE NOT NULL,
  bill_status     ENUM('Paid','Pending','Overdue') NOT NULL DEFAULT 'Pending',
  FOREIGN KEY (flat_id) REFERENCES flats(flat_id)
);

CREATE TABLE payments (
  payment_id      INT AUTO_INCREMENT PRIMARY KEY,
  bill_id         INT NOT NULL,
  resident_id     INT NOT NULL,
  payment_date    DATE NOT NULL,
  amount          DECIMAL(10,2) NOT NULL,
  payment_method  ENUM('UPI','Card','Net Banking','Cheque') NOT NULL,
  transaction_id  VARCHAR(50) NOT NULL UNIQUE,
  payment_status  ENUM('Successful','Pending','Failed') NOT NULL,
  FOREIGN KEY (bill_id) REFERENCES maintenance_bills(bill_id),
  FOREIGN KEY (resident_id) REFERENCES residents(resident_id)
);

```

**Normalization.** The schema is in third normal form. Every non-key column depends only on its table's primary key: block and floor information lives only in flats; resident contact details live only in residents; amounts and due dates live only in maintenance\_bills. Derived values—such as total pending dues, occupancy percentage, or the number of visitors currently inside—are calculated when needed rather than stored, which avoids inconsistencies.

## CHAPTER -6 IMPLEMENTATION

### 6.1 Module-Wise Implementation

The source code is organized into the following folders under `src/`:

|                             |                                                                                                                                                                                                                                                                             |
|-----------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>src/</code>           |                                                                                                                                                                                                                                                                             |
| ├─ <code>main.jsx</code>    | Entry point - mounts <code>&lt;App/&gt;</code> inside React StrictMode                                                                                                                                                                                                      |
| ├─ <code>App.jsx</code>     | Route definitions                                                                                                                                                                                                                                                           |
| ├─ <code>index.css</code>   | Global styles, layout grid, responsive rules                                                                                                                                                                                                                                |
| ├─ <code>components/</code> | <code>DataTable</code> , <code>RecordForm</code> , <code>Modal</code> , <code>Navbar</code> , <code>Sidebar</code> , <code>ProtectedRoute</code> , <code>StatCard</code> , <code>StatusBadge</code> , <code>Toast</code> , <code>EmptyState</code> , <code>RoleBadge</code> |
| ├─ <code>layouts/</code>    | <code>DashboardLayout</code> (sidebar + navbar + page outlet + toast)                                                                                                                                                                                                       |
| ├─ <code>pages/</code>      | <code>Dashboard</code> , <code>RoleDashboard</code> , <code>ResourcePage</code> , <code>Notices</code> , <code>Amenities</code> , <code>Settings</code> , <code>Login</code> , and thin pages for each resource                                                             |
| ├─ <code>config/</code>     | <code>resources.js</code> (module definitions), <code>permissions.js</code> (role rules)                                                                                                                                                                                    |
| ├─ <code>context/</code>    | <code>AppContext.jsx</code> (provider), <code>AppStateContext.js</code> (useApp hook)                                                                                                                                                                                       |
| ├─ <code>services/</code>   | <code>api.js</code> (data service), <code>auth.js</code> (sign-in and session)                                                                                                                                                                                              |
| └─ <code>data/</code>       | <code>mockData.js</code> (sample records matching MySQL columns)                                                                                                                                                                                                            |

Each functional module is described below.

#### 1. Authentication module

Implemented in `pages/Login.jsx` and `services/auth.js`. The login screen has a branded left panel and a sign-in form with a *Demo Role* selector that fills the matching credentials automatically, an email field, a password field with a show/hide toggle, and an error message area. On submit, `signIn` looks up the account by case-insensitive email and password, removes the password from the user object, stores the user in `localStorage`, and navigates to the dashboard.

#### 2. Dashboard module

`pages/Dashboard.jsx` renders the administrator dashboard: six summary cards (Total Flats, Occupied Flats, Total Residents, Pending Maintenance in rupees, Open Complaints, Active Visitors), an occupancy donut chart, a maintenance-status bar chart, a complaint-status chart, a recent-activity feed, and lists of recent visitors and payments. For every other role it delegates to `RoleDashboard.jsx`, which defines a tailored set of four cards, quick-action buttons, and a list of relevant records for each of the seven remaining roles. For example, the Treasurer sees Total Collection, Pending Dues, Overdue Bills, and Successful Payments, while Security sees Visitors Today, Currently Inside, Expected, and Exited.

#### 3. Flats module

Lists every flat with ID, number, block, floor, type, and an occupancy badge. Filters are provided for block, floor, and occupancy. Administrators can add, edit, and delete flats; Committee, Treasurer, and Secretary have view access.

#### **4. Residents module**

Shows each resident with an initials avatar, email, phone, flat (displayed as *Block-Number*, e.g. A- 101, by looking up the related flat), resident type, and role. Summary cards count total residents, owners, and tenants. The add/edit form uses a dropdown populated from the flats table for the flat\_id field.

#### **5. Visitors module**

Records visitor name, phone, purpose, destination flat, entry and exit times, and status (Inside, Exited, Expected). Security staff, the Committee, the Secretary, and the Admin manage all visitor entries; residents see and pre-register only visitors for their own flat, and the flat is filled in automatically.

#### **6. Staff module**

Maintains society employees with name, phone, type (Security, Housekeeping, Maintenance, Gardener), and address. Summary cards show the count per department.

#### **7. Maintenance (bills) module**

Tracks monthly bills per flat with month, amount in rupees, due date, and status (Paid, Pending, Overdue). The Treasurer and Admin create and update bills; residents see only their own flat's bills.

#### **8. Payments module**

Records payments against a bill and a resident, with date, amount, method (UPI, Card, Net Banking, Cheque), transaction ID, and status. The *Total Collection* card sums the amounts of successful payments and formats the total in Indian Rupees.

#### **9. Complaints module**

Residents raise complaints with a title, description, and date; the resident is attached automatically. The Committee, Secretary, Maintenance Staff, and Admin update the status from Open to In Progress to Resolved. Clicking a row opens a detail dialog showing the full description.

#### **10. Notices module**

Implemented as a card grid in pages/Notices.jsx. Each notice shows its date, title, and description with View, Edit, and Delete actions according to role. All roles can read notices, which makes this the society's official announcement board.

#### **11. Amenities module**

Implemented as a visual card grid in pages/Amenities.jsx. Each card shows the amenity's icon, name, location, and availability. Residents see a *Book* button that is disabled when the amenity is

under maintenance; managers can add and edit amenities.

## **12. Amenity bookings module**

Records which resident booked which amenity, on what date and time, with status Confirmed, Pending, Completed, or Cancelled. Summary cards show today's bookings, upcoming bookings, completed bookings, and cancellations.

## **13. Settings module**

Shows the signed-in user's profile—initials avatar, name, email, account type, and role—with a form to update contact details.

## **6.2 Front-End Logic (UI Rendering, State Management)**

### ***Global state with React Context***

All application data is held in one React Context so that every page shares the same records. When the provider mounts, it requests all ten resources in parallel with `Promise.all`, stores them in state, and clears the loading flag. Mutation functions call the service and then update state immutably, which causes every component that uses the data to re-render.

```
export function AppProvider({ children }) {
  const [data, setData] = useState(Object.fromEntries(resources.map((key) => [key, []])))
  const [loading, setLoading] = useState(true)
  const [toast, setToast] = useState('')

  useEffect(() => {
    Promise.all(resources.map((resource) => api.list(resource))).then((results) => {
      setData(Object.fromEntries(resources.map((resource, index) => [resource, results[index]])))
      setLoading(false)
    })
  }, [])

  const updateRecord = async (resource, id, values, idField) => {
    const record = await api.update(resource, id, values)
    setData((current) => ({ ...current,
      [resource]: current[resource].map((item) => Number(item[idField]) === Number(id)?
        record : item })))
    notify('Changes saved successfully.')
  }
  ...
}
```

Components access this state through a small custom hook, `useApp()`, defined in `AppStateContext.js`. Keeping the context object in a separate file allows React's fast refresh to work correctly during development.

## Configuration-driven rendering

The generic ResourcePage receives a resource name, reads its configuration, and derives everything it needs. The rows displayed go through three stages: **scoping** by role (scopeRecords), **searching** across every field value and the related flat label, and **filtering** by the selected dropdown values. Both stages are wrapped in useMemo so they are recalculated only when the data, the query, or the filters change.

```
const scopedRows = useMemo(() => scopeRecords(user, resource, data[resource] || []),
[user, resource, data])

const rows = useMemo(() => scopedRows.filter((row) => { const
queryMatch = Object.values(row).some((value) =>
  String(value).toLowerCase().includes(query.toLowerCase())) || /* flat label match */
  return queryMatch && Object.entries(filters).every(([key, value]) => !value ||
String(row[key]) === value)
}), [data.flats, scopedRows, query, filters])
```

## Data table

DataTable handles sorting, pagination, and cell rendering. Clicking a column header toggles ascending and descending order; sorting uses localeCompare with the numeric option, so that "Flat 10" sorts after "Flat 9". The table shows six records per page with previous and next controls. Each cell is rendered according to the column type in the configuration: badge renders a coloured StatusBadge; currency formats rupees; date and datetime use Indian date formats; and flat, resident, and amenity replace a foreign-key number with a readable label by looking up the related table.

## Forms and validation

RecordForm generates its inputs from the field list. Fields with fixed options become dropdowns, and fields of type flat, resident, amenity, or bill become dropdowns populated from the related table, so users cannot enter an invalid foreign key. On submit, the form checks required fields, email format, and phone format, and shows an error under each invalid field.

```
fields.forEach((field) => {
  if (field.required && !String(values[field.name]).trim())
    nextErrors[field.name] = `${field.label} is required.`
  if (field.type === 'email' && values[field.name] &&
    !/^S+@\S+\.\S+$/i.test(values[field.name]))
    nextErrors[field.name] = 'Enter a valid email address.'
  if (field.type === 'tel' && values[field.name] &&
    !/^d{10,15}$/i.test(values[field.name]))
    nextErrors[field.name] = 'Enter a 10 to 15 digit phone number.'
})
```

Before saving, ResourcePage normalizes the values: any field ending in `_id`, as well as `amount` and `floor_number`, is converted from the string returned by the input into a number, matching the integer and decimal columns in MySQL.

### ***Navigation, search, and feedback***

The Sidebar groups links into sections and hides any link the current role cannot open, and hides empty sections entirely. The Navbar provides a global search that, after two characters, searches residents, flats, visitors, complaints, bills, and notices (limited to pages the user may open) and shows up to seven results. A Toast confirms every successful add, update, and delete for 2.6 seconds. Dashboards show a skeleton placeholder and module pages show a spinner while data is loading.

## **6.3 API Integration**

All data access is centralized in `src/services/api.js`. The service keeps a working copy of the sample data and exposes four generic asynchronous operations. A small wait helper resolves each result after 180 ms to imitate network latency, so loading states can be seen and tested.

```
const idFields = { flats: 'flat_id', residents: 'resident_id', visitors: 'visitor_id',
  staff: 'staff_id',
  maintenance: 'bill_id', payments: 'payment_id', complaints: 'complaint_id', notices:
  'notice_id',
  amenities: 'amenity_id', bookings: 'booking_id' }
const wait = (value) => new Promise((resolve) => setTimeout(() => resolve(value), 180))

async function add(resource, values) {
  const idField = idFields[resource]
  const nextId = Math.max(0, ...store[resource].map((row) => Number(row[idField]))) + 1
  const record = { ...values, [idField]: nextId }
  store[resource] = [record, ...store[resource]]
  return wait(record)
}

export const api = { list, add, update, remove }
// Named helpers are generated for each resource: getFlats, addFlat, updateFlat,
deleteFlat, ...
```

Because every page calls only the context, and the context calls only these four functions, connecting the real backend means replacing their bodies with HTTP requests. The planned version is shown below; the base URL comes from a Vite environment variable, and the JWT token is sent in the Authorization header.

```

const BASE = import.meta.env.VITE_API_URL // e.g.
https://api.havensociety.in/api/v1
const headers = () => ({ 'Content-Type': 'application/json',
                        Authorization: `Bearer ${localStorage.getItem('haven-token')}`
})

async function request(method, path, body) {
  const res = await fetch(`${BASE}/${path}`, { method, headers: headers(), body: body &&
  JSON.stringify(body) })
  if (!res.ok) throw new Error((await res.json()).message || res.statusText)
  return res.status === 204 ? { success: true } : res.json()
}

const list      = (resource)           => request('GET',      resource)
const add       = (resource, values)    => request('POST',    resource, values)
const update    = (resource, id, values) => request('PUT',    `${resource}/${id}`, values)
const remove    = (resource, id)        => request('DELETE',  `${resource}/${id}`)

```

The resource names map to REST endpoints and database tables as follows:

| Frontend route | Service resource | REST endpoint       | MySQL table       |
|----------------|------------------|---------------------|-------------------|
| /flats         | flats            | /api/v1/flats       | flats             |
| /residents     | residents        | /api/v1/residents   | residents         |
| /visitors      | visitors         | /api/v1/visitors    | visitors          |
| /staff         | staff            | /api/v1/staff       | staff             |
| /maintenance   | maintenance      | /api/v1/maintenance | maintenance_bills |
| /payments      | payments         | /api/v1/payments    | payments          |
| /complaints    | complaints       | /api/v1/complaints  | complaints        |
| /notices       | notices          | /api/v1/notices     | notices           |
| /amenities     | amenities        | /api/v1/amenities   | amenities         |
| /bookings      | bookings         | /api/v1/bookings    | amenity_bookings  |

## 6.4 Authentication & Authorization

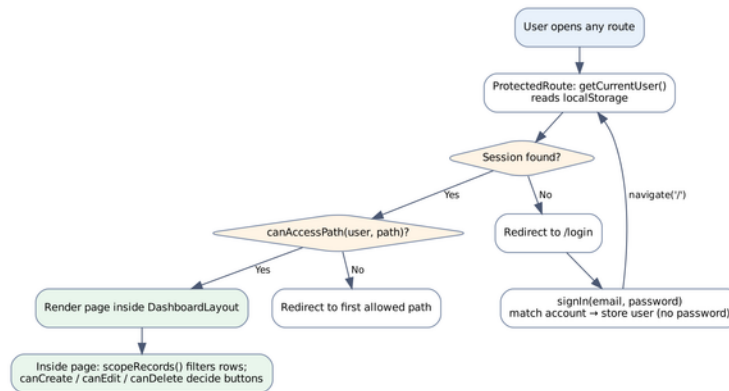


Figure 6.1 – Authentication and authorization flow

### Authentication

The current release includes eight demonstration accounts, one per role, defined in auth.js. Signing in stores a copy of the account without its password under the localStorage key haven-demo-user; signing out removes it. getCurrentUser reads and parses this value safely inside a try...catch block, so corrupted storage simply returns no user. Each account also carries the identifiers needed for scoping: residents carry resident\_id and flat\_id, and staff carry staff\_id.

### Authorization

Authorization rules are defined in one file, src/config/permissions.js, and applied at three levels:

1. **Route level** – canAccessPath(user, path) decides whether a page may be opened; ProtectedRoute redirects otherwise, and the sidebar hides links the user cannot open.
2. **Action level** – canCreateResource, canEditRecord, and canDeleteRecord decide whether the Add, Edit, and Delete buttons are shown for each page and each row.
3. **Record level** – scopeRecords filters the rows a user sees, and ownsRecord decides whether a resident may edit or delete a particular visitor, complaint, or booking.

```

export function scopeRecords(user, resource, rows) {
  if (!user || !rows) return []
  if (user.role === 'Resident') {
    if (['visitors', 'maintenance'].includes(resource))
      return rows.filter((row) => Number(row.flat_id) === Number(user.flat_id))
    if (['payments', 'complaints', 'bookings'].includes(resource))
      return rows.filter((row) => Number(row.resident_id) === Number(user.resident_id))
  }
  if (user.accountType === 'staff' && resource === 'staff')
    return rows.filter((row) => Number(row.staff_id) === Number(user.staff_id))
  return rows
}

```

The resulting permission matrix is shown below (**M** = view and manage, **V** = view only, **O** = own records only, with create/edit/delete on own records where noted, – = no access).

| Module      | Admin | Committee | Treasurer | Secretary | Resident               | Security | House-keeping | Maint. Staff |
|-------------|-------|-----------|-----------|-----------|------------------------|----------|---------------|--------------|
| Dashboard   | Admin | Role      | Role      | Role      | Role                   | Role     | Role          | Role         |
| Flats       | M     | V         | V         | V         | –                      | –        | –             | –            |
| Residents   | M     | M*        | V         | V         | –                      | –        | –             | –            |
| Visitors    | M     | M         | –         | M         | O<br>(add/edit/delete) | M*       | –             | –            |
| Staff       | M     | V         | –         | –         | –                      | –        | –             | –            |
| Maintenance | M     | V         | M*        | –         | O (view)               | –        | –             | –            |
| Payments    | M     | V         | M*        | –         | O (view)               | –        | –             | –            |
| Complaints  | M     | M         | –         | M         | O                      | –        | –             | M*           |
| Notices     | M     | M         | V         | M         | (add/edit/delete)      | V        | V             | V            |
| Amenities   | M     | M*        | –         | M*        | V                      | –        | –             | –            |
| Bookings    | M     | M         | –         | M         | V + Book<br>O          | –        | –             | –            |
| Settings    | ✓     | ✓         | ✓         | ✓         | (add/edit/delete)      | ✓        | ✓             | ✓            |

\* Can add and edit but not delete. Only the Admin can delete flats, residents, staff, bills, payments, and amenities; the Committee and Secretary can also delete visitors, complaints, notices, and bookings.

**Security note.** In the current frontend-only release, these checks control what the interface displays. They are not a substitute for server-side security, because a determined user can modify code running in their own browser. When the backend is connected, the same rules will be enforced in Express middleware on every request, passwords will be stored as bcrypt hashes instead of plain text, and the session will be a signed JWT rather than a user object in localStorage.

## CHAPTER 7- FEATURES

### 7.1 List of Main Features

| #  | Feature                        | Summary                                                                                                                                                           |
|----|--------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1  | Role-based sign-in             | Eight demo accounts with a role selector that fills credentials; invalid logins show a clear error.                                                               |
| 2  | Admin analytics dashboard      | Six KPI cards, occupancy donut chart, maintenance and complaint bar charts, activity feed, recent visitors and payments.                                          |
| 3  | Role-specific dashboards       | Separate cards, quick actions, and record lists for Resident, Treasurer, Secretary, Committee, Security, Housekeeping, and Maintenance Staff.                     |
| 4  | Role-filtered navigation       | Sidebar shows only permitted modules, grouped into Overview, Society, Gate, Finance, Community, and Account.                                                      |
| 5  | Flat and occupancy management  | Flats with block, floor, type, and occupancy, filterable by block, floor, and occupancy.                                                                          |
| 6  | Resident directory             | Residents linked to flats, with owner/tenant type and portal role; owner and tenant counts.                                                                       |
| 7  | Visitor gate register          | Entry/exit times and Inside, Expected, and Exited status; residents pre-register guests for their own flat.                                                       |
| 8  | Staff records                  | Security, housekeeping, maintenance, and gardening staff with contact details and counts per department.                                                          |
| 9  | Maintenance billing            | Monthly bills per flat with amount, due date, and Paid/Pending/Overdue status.                                                                                    |
| 10 | Payment tracking               | Payments linked to bills and residents with method, transaction ID, status, and total collection.                                                                 |
| 11 | Complaint management           | Residents raise complaints; staff and management move them from Open to In Progress to Resolved.                                                                  |
| 12 | Notice board                   | Card-based announcements visible to every role; managers create, edit, and delete.<br>Amenity cards with availability; bookings with date, time slot, and status. |
| 1  | Amenities and bookings         | Every table supports text search, dropdown filters, column sorting, and six-row pages.                                                                            |
| 3  | Search, filter, sort, paginate | Top-bar search across residents, flats, visitors, complaints, bills, and notices, limited to permitted pages.                                                     |
| 15 | Global search                  | Foreign keys chosen from dropdowns of existing flats, residents, bills, and amenities; validation for required fields, email, and phone.                          |
| 16 | Relationship-aware forms       | Residents see only their own flat's bills and visitors and their own payments, complaints, and bookings.                                                          |
| 17 | Record-level privacy           |                                                                                                                                                                   |

|    |                           |                                                                                           |
|----|---------------------------|-------------------------------------------------------------------------------------------|
| 18 | Feedback and confirmation | Success toasts, delete confirmation dialogs, loading spinners, and dashboard skeletons.   |
| 1  | Indian localization       | Amounts in ₹ with Indian digit grouping; dates in the en-IN format.                       |
| 9  | Responsive design         | Layouts adapt from wide desktop screens to 360-pixel-wide phones with an off-canvas menu. |

## 7.2<sup>0</sup> How Each Feature Works (User Perspective + Technical Description)

### 7.2.1 Role-based sign-in

**User perspective:** The user opens the portal and sees the sign-in page. They either choose a role from the *Demo Role* list—for example "Resident · Rahul Kumar"—which fills the email and password, or they type their credentials. After pressing *Sign in* they land on their own dashboard. A wrong password shows the message "Email or password is incorrect."

**Technical description:** The role selector's `onChange` handler finds the demo account by ID and sets the email and password state. On submit, `signIn()` searches the account list; on success it stores the user (without password) in `localStorage` and calls `navigate('/')`. `ProtectedRoute` reads the session on every navigation.

### 7.2.2 Admin analytics dashboard

**User perspective:** The administrator sees the society's health at a glance: 12 flats, 9 occupied (75 % occupancy), 10 residents, ₹36,000 of pending maintenance across 6 bills, open complaints, and visitors currently inside. Charts show occupancy, bill status, and complaint progress, and lists show the latest visitors and payments. A button opens the maintenance page.

**Technical description:** All figures are derived from the context data at render time. For example, pending maintenance is the sum of amount for bills whose status is not Paid, formatted with `Intl.NumberFormat('en-IN', {style: 'currency', currency: 'INR'})`. Chart data arrays are built by counting records per status and passed to `Recharts` components inside `ResponsiveContainer` so they resize with the panel.

### 7.2.3 Role-specific dashboards

**User perspective:** Every non-admin user sees a dashboard written for their job. Rahul (Resident) sees My Flat A-101, Amount Due, My Complaints, and My Bookings, plus quick actions to pay maintenance, raise a complaint, book an amenity, or register a visitor. The security guard sees gate counts; the treasurer sees collections and dues.

**Technical description:** `RoleDashboard` holds a configuration object keyed by role name; each entry defines a subtitle, four card tuples, a list of quick actions (label, path, icon), a list title, and a records

array. The component selects `roleContent[user.role]` and renders it with the shared `StatCard` and `StatusBadge` components.

#### **7.2.4 Managing records (*Flats, Residents, Visitors, Staff, Bills, Payments, Complaints, Bookings*)**

**User perspective:** Each module page has the same familiar layout: a title with an *Add* button, summary cards, a search box with filters, and a table. Clicking *Add* opens a form in a dialog; clicking the pencil icon edits a row; the eye icon or clicking the row shows details; and the bin icon asks for confirmation before deleting. A green toast confirms every change.

**Technical description:** `ResourcePage` keeps local state for the query, filters, and the record being edited, viewed, or deleted. Buttons are shown only when the corresponding permission function returns true. Saving calls `addRecord` or `updateRecord` from the context; deleting calls `deleteRecord`. The table re-renders automatically when the context state changes.

#### **7.2.5 Visitor gate register**

**User perspective:** When a guest arrives, the security guard adds a visitor with name, phone, purpose, destination flat, entry time, and status *Inside*. When the guest leaves, the guard edits the entry, adds the exit time, and sets status to *Exited*. A resident expecting a guest can pre-register them as *Expected*; the flat is filled in automatically.

**Technical description:** Entry and exit use `datetime-local` inputs. For residents, the `flat_id` field is removed from the form and `save()` merges `flat_id: user.flat_id` into the values before saving. The Security dashboard counts visitors by status and by entry date.

#### **7.2.6 Maintenance billing and payments**

**User perspective:** The treasurer creates a bill for a flat with month, amount, and due date. When a resident pays, the treasurer records the payment against that bill with method and transaction ID. Residents open *Maintenance* and *Payments* to see only their own bills and receipts.

**Technical description:** The payment form's *Bill* dropdown lists bills as "Bill #id · ₹amount", and the *Resident* dropdown lists residents by name. `scopeRecords` filters bills by `flat_id` and payments by `resident_id` for residents. The *Total Collection* card sums successful payment amounts.

#### **7.2.7 Complaint lifecycle**

**User perspective:** A resident raises a complaint such as "Lift not working" with a description. Maintenance staff see it in their queue and change its status to *In Progress* and then *Resolved*. The resident sees the status change on their dashboard.

**Technical description:** For residents, the `resident_id` field is hidden and filled automatically. Maintenance Staff have `manage` permission for complaints only. Status values are rendered as

coloured badges, and the description column is truncated in the table with the full text available in the detail dialog.

### 7.2.8 Notices and amenities

**User perspective:** Everyone sees notices as cards with date, title, and description. The secretary publishes new notices such as "Water Supply Maintenance". On the Amenities page, residents see each facility's location and availability and press *Book* to go to bookings; the button is disabled for amenities under maintenance.

**Technical description:** These pages use the same RecordForm and Modal components with their own field lists. Dates are formatted with `toLocaleDateString('en-IN', {day, month: 'long', year})`. The Book button's disabled attribute is bound to `availability_status !== 'Available'`.

### 7.2.9 Search and global search

**User perspective:** Typing in a page's search box narrows the table immediately; typing "A-101" finds records for that flat even though the table stores only a flat ID. The search bar at the top of every page finds residents, flats, visitors, complaints, bills, and notices and jumps to the right page.

**Technical description:** Page search checks every field value and also the composed *Block-Number* label of the related flat. Global search starts after two characters, scans six resource/field pairs, keeps only those the user may access, and returns up to two matches per group and seven in total.

### 7.2.10 Responsive layout

**User perspective:** On a phone the sidebar is hidden and a menu button opens it as an overlay; tapping outside closes it. Cards stack vertically and tables scroll horizontally.

**Technical description:** The layout uses CSS Grid with media queries. DashboardLayout holds a sidebarOpen state; the Navbar's menu button sets it to true, and both the close button and the overlay set it to false. Navigating via a link also closes the menu.

## CHAPTER -8 TESTING

Testing was carried out to confirm that each module behaves correctly, that role permissions are applied consistently, and that the application is ready to be connected to a backend. Four kinds of verification were used: static analysis and build checks, API contract testing preparation with Postman, end-to-end integration testing in a real browser, and manual exploratory testing on desktop and mobile layouts.

### 8.1 Postman Testing (Frontend/Backend)

Postman is used to test HTTP APIs by sending requests and checking the status code and response body. Because the current release uses an in-memory service layer instead of a running server, the Postman work in this phase focused on defining and documenting the REST contract that the backend must satisfy, so that the backend can be verified immediately once it is built. The contract was derived directly from the service functions (list, add, update, remove) and the column names used by the frontend.

A Postman collection named *Haven Society API* is organized into one folder per resource, with a collection variable `{{baseUrl}}` (for example `http://localhost:5000/api/v1`) and a `{{token}}` variable that the login request sets automatically through a test script. The core requests and the expected results are:

| Method | Endpoint        | Request / purpose                                      | Expected response                         |
|--------|-----------------|--------------------------------------------------------|-------------------------------------------|
| POST   | /auth/login     | Body { email, password }                               | 200 + JWT and user; 401 on wrong password |
| GET    | /flat           | List all flats (Bearer token)                          | 200 + JSON array                          |
| POST   | s<br>/flat      | Create flat with number, block, floor, type, occupancy | 201 + new record with flat_id             |
| PUT    | /flats/:id<br>s | Update a flat                                          | 200 + updated record; 404 if not found    |
| DELETE | /flats/:id      | Delete a flat (Admin only)                             | 204; 403 for other roles                  |
| GET    | /maintenance    | As Resident: only own flat's bills                     | 200 + filtered array                      |
| POST   | /payments       | Record payment for bill_id and resident_id             | 201; 400 if bill does not exist           |
| POST   | /complaints     | As Resident: resident_id set from token                | 201 + record linked to caller             |

|      |                 |                                     |                                |
|------|-----------------|-------------------------------------|--------------------------------|
| PUT  | /complaints/:id | As Maintenance Staff: change status | 200; 403 if role not permitted |
| POST | /residents      | Missing email or 8-digit phone      | 400 + validation messages      |
| GET  | /payments       | No token                            | 401 Unauthorized               |

Each request carries Postman test scripts that assert the status code, the response time, and the shape of the JSON, for example:

```
pm.test("Status is 201", () => pm.response.to.have.status(201));
pm.test("Record has an id", () =>
pm.expect(pm.response.json().complaint_id).to.be.a("number"));
pm.test("Linked to signed-in resident", () => {
  const expected = pm.collectionVariables.get("residentId");
  pm.expect(pm.response.json().resident_id).to.eql(expected);
});
```

On the frontend side, the equivalent behaviour of the service layer was verified through the browser tests in Section 8.4: records are created with the next available ID, updates replace only the targeted record, deletions remove it, and lists return the current records. The Postman collection will be executed in full against the Express API in the next phase.

## 8.2 Integration Testing

Integration testing checks that the separate parts of the application work together: routing, permission checks, pages, shared components, the context, and the service layer. These tests were automated with **Playwright**, which drives a real Chromium browser against the running Vite development server, exactly as a user would—typing into fields, clicking buttons, and reading what appears on screen.

The integration scenarios covered complete user journeys rather than single functions:

- **Authentication → routing → layout:** an unauthenticated user is redirected to login; a valid login reaches the correct dashboard; logout clears the session.
- **Permissions → sidebar → route guard:** a resident sees only permitted links, and typing a forbidden URL redirects to the dashboard.
- **Form → context → service → table:** adding a flat increases the record count and shows a toast; editing a complaint's status updates the badge in the table.
- **Scoping across modules:** a resident's maintenance page shows only flat A-101, and a complaint they create is linked to their account without them choosing a resident.

- **Cross-role behaviour:** Security can manage visitors but cannot open payments; the Treasurer can view residents without edit buttons; the Secretary can delete a notice.
- **Responsive integration:** on a 390 × 844 viewport the menu button opens the sidebar overlay.

Each scenario starts in a fresh browser context so that one test's session does not affect another's.

### 8.3 Tools Used for Testing

| Tool                  | How it was used                                                                                                                           |
|-----------------------|-------------------------------------------------------------------------------------------------------------------------------------------|
| ESLint 10             | Static analysis of all source files with React Hooks rules; npm run lint completes with 0 errors and 0 warnings.                          |
| Vite build            | npm run build compiles 2,458 modules without errors, confirming that all imports resolve and the production bundle can be generated.      |
| Playwright (Chromium) | Automated end-to-end and integration tests across all roles; also used to capture the screenshots in Chapter 14.                          |
| Postman               | REST API contract and test scripts for the planned backend (Section 8.1).                                                                 |
| Chrome DevTools       | Console monitoring, React component inspection, network-throttling checks of loading states, and device emulation for responsive testing. |
| Manual testing        | Exploratory testing of each page and form on desktop and mobile layouts.                                                                  |

### 8.4 Test Cases and Results

The table below lists the test cases executed with Playwright against the current release, together with the actual observed results. 21 of 23 test cases passed. The two failures are genuine findings that are documented as known issues in Section 10.3.

| ID    | Test case                          | Steps                                        | Expected result                                 | Actual result                                                                                           | Status |
|-------|------------------------------------|----------------------------------------------|-------------------------------------------------|---------------------------------------------------------------------------------------------------------|--------|
| TC-01 | Unauthenticated access is blocked  | Open /residents without signing in           | Redirect to /login                              | Redirected to /login                                                                                    | Pass   |
| TC-02 | Login with wrong password          | Enter admin email with an incorrect password | Error message, user stays on login page         | Error message "Email or password is incorrect..." shown                                                 | Pass   |
| TC-03 | Admin login with valid credentials | Sign in as admin@havenwoods.in / admin123    | Admin dashboard with summary cards and charts   | Dashboard loaded: Good morning, Admin                                                                   | Pass   |
| TC-04 | Required-field validation          | Open Add Resident and submit the empty form  | Error shown under every required field          | 7 field errors shown (7 required fields)                                                                | Pass   |
| TC-05 | Phone number validation            | Enter "12ab" as phone and submit             | Form is not submitted; invalid phone is flagged | Browser blocked submission with "Please match the requested format." (pattern check); no record created | Pass   |
| TC-06 | Search residents table             | Type "Rahul" in the Residents search box     | Only matching residents are listed              | 1 row shown (Rahul Kumar)                                                                               | Pass   |

|       |                                           |                                                   |                                                   |                                                                                                                   |      |
|-------|-------------------------------------------|---------------------------------------------------|---------------------------------------------------|-------------------------------------------------------------------------------------------------------------------|------|
| TC-07 | Filter flats by occupancy                 | Choose "Vacant" in the occupancy filter           | Only vacant flats are listed (3 in sample data)   | 3 vacant flats listed                                                                                             | Pass |
| TC-08 | Create a flat record                      | Add Flat D-501, 3BHK, Vacant                      | Record added, success toast, total becomes 13     | Toast "Flat added successfully."; footer "Showing 6 of 13 records"                                                | Pass |
| TC-09 | Success message wording when adding staff | Add a staff member                                | Toast reads "Staff member added successfully."    | Toast reads "Staf added successfully." (known bug, see 10.3)                                                      | Fail |
| TC-10 | Update complaint status                   | Edit first complaint and set status to Resolved   | Status badge changes to Resolved                  | Badge now shows "Resolved"                                                                                        | Pass |
| TC-11 | Global search                             | Type "Water" in the top search bar                | Matching complaints and notices listed            | 2 results (complaint + notice)                                                                                    | Pass |
| TC-12 | Logout                                    | Open profile menu and choose Logout               | Session removed and login page shown              | Returned to /login; session key cleared                                                                           | Pass |
| TC-13 | Resident navigation is role-scoped        | Sign in as Rahul Kumar (Resident)                 | No Flats, Residents or Staff links                | Links: Dashboard, Visitors, Maintenance, Payments, Complaints, Notices, Amenities, Bookings, Settings             | Pass |
| TC-14 | Resident blocked from admin route         | Type /residents in the address bar                | Redirect to dashboard                             | Redirected to /                                                                                                   | Pass |
| TC-15 | Resident sees only own bills              | Open Maintenance as Rahul (Flat A-101)            | Only bills for A-101                              | 1 bill shown, flat A-101                                                                                          | Pass |
| TC-16 | Resident cannot create bills              | Look for Add Bill button                          | Button hidden                                     | Add Bill button not rendered                                                                                      | Pass |
| TC-17 | Resident raises a complaint               | Add complaint without choosing a resident         | Complaint saved and linked to Rahul automatically | Resident field hidden; Rahul now has 2 complaints                                                                 | Pass |
| TC-18 | Security cannot open finance pages        | Sign in as Security and open /payments            | Redirect to dashboard                             | Redirected to /                                                                                                   | Pass |
| TC-19 | Security can register visitors            | Open Visitors as Security                         | Add Visitor button available                      | Add Visitor button shown                                                                                          | Pass |
| TC-20 | Treasurer has read-only resident access   | Open Residents as Treasurer                       | List visible, no Add/Edit/Delete                  | Table shown with View action only                                                                                 | Pass |
| TC-21 | Secretary deletes a notice                | Delete first notice and confirm                   | Notice removed from list                          | Notices: 5 → 4                                                                                                    | Pass |
| TC-22 | Responsive layout on mobile               | Open dashboard at 390×844 and tap the menu button | Sidebar collapses to a slide-in menu              | Sidebar opened as overlay menu                                                                                    | Pass |
| TC-23 | Data persistence after reload             | Add a flat, then refresh the browser              | Record still present                              | Footer after reload: "Showing 6 of 12 records" (in-memory mock data resets — expected until backend is connected) | Fail |

### Summary of results.

| Category                 | Executed  | Passed    | Failed   |
|--------------------------|-----------|-----------|----------|
| Authentication & session | 4         | 4         | 0        |
| Form validation          | 2         | 2         | 0        |
| CRUD, search & filters   | 7         | 6         | 1        |
| Role-based access        | 8         | 8         | 0        |
| Responsive UI            | 1         | 1         | 0        |
| Data persistence         | 1         | 0         | 1        |
| <b>Total</b>             | <b>23</b> | <b>21</b> | <b>2</b> |

The two failed cases do not block normal use. TC-09 is a wording defect in the success message, and TC-23 is the expected consequence of using an in-memory data store; it will pass once the service layer is connected to the MySQL database through the backend API.

## CHAPTER -9 DEPLOYMENT

### 9.1 Steps to Deploy

The Society Management Portal is a static single-page application, so deploying the current release means building the project and publishing the resulting files to a web host. The steps below take the project from source code to a public HTTPS address.

#### Step 1 – Prepare the project locally

```
git clone https://github.com/<your-account>/society-management-frontend.git
cd society-management-frontend
npm install          # installs React, Vite, React Router, Recharts, Lucide, ESLint
npm run lint         # must report no problems
npm run build        # creates the dist/ folder
```

The build prints the output files and their sizes, for example:

```
vite v8.2.2 building client environment for production...
✓ 2458 modules transformed.
dist/index.html                0.53 kB | gzip:    0.31 kB
dist/assets/index-DcLhAtAm.css 25.85 kB | gzip:    6.21 kB
dist/assets/index-BG1OJBxa.js  686.08 kB | gzip: 201.95 kB
✓ built in 1.77s
```

#### Step 2 – Preview the production build

Run `npm run preview` and open the address shown (by default `http://localhost:4173`). This serves the exact files that will be deployed, which helps catch problems that do not appear in development mode.

#### Step 3 – Push the code to GitHub

Commit the source files and push to the main branch. The `dist` folder and `node_modules` are excluded by `.gitignore`; the hosting service builds the project itself.

#### Step 4 – Deploy the frontend (Vercel)

1. Sign in to Vercel with the GitHub account and choose *Add New* → *Project*.
2. Import the `society-management-frontend` repository. Vercel detects Vite automatically.
3. Confirm the settings: build command `npm run build`, output directory `dist`, install command `npm install`.
4. Click *Deploy*. After about a minute the portal is live at a `*.vercel.app` address with HTTPS.
5. Add a `vercel.json` rewrite so that client-side routes such as `/payments` work when refreshed:

```
{ "rewrites": [ { "source": "/(.*)", "destination": "/index.html" } ] }
```

**Alternative – Netlify:** set the build command to `npm run build` and the publish directory to `dist`, and add a `public/_redirects` file containing `/* /index.html 200`.

### ***Step 5 – Continuous deployment***

After the first deployment, every push to main triggers a new build and deployment automatically, and every pull request receives its own preview URL for review before merging.

### ***Step 6 – Deploy the backend and database (next phase)***

1. Create a managed MySQL 8 database and run the schema script to create the ten tables in `society_management`.
2. Deploy the Express API to a Node.js host (Render, Railway, or AWS EC2) and set its environment variables.
3. Set `VITE_API_URL` in the frontend project settings to the API's public URL and redeploy.
4. Run the Postman collection against the live API and the Playwright suite against the live site.

## **9.2 Environment Configuration**

Configuration values that differ between development and production are kept out of the source code and supplied as environment variables. Vite exposes only variables prefixed with `VITE_` to browser code, which prevents server secrets from being bundled into the frontend by mistake.

| Variable                      | Where    | Purpose / example                                                                                                                   |
|-------------------------------|----------|-------------------------------------------------------------------------------------------------------------------------------------|
| VITE_API_URL                  | Frontend | Base URL of the REST API, e.g.<br><code>http://localhost:5000/api/v1</code> in development and the<br>hosted API URL in production. |
| VITE_APP_NAME                 | Frontend | Display name of the society (default "Haven Society").                                                                              |
| PORT                          | Backend  | Port the Express server listens on (e.g. 5000).                                                                                     |
| DB_HOST, DB_PORT              | Backend  | MySQL server address and port (3306).                                                                                               |
| DB_USER,<br>DB_PASSWORD       | Backend  | Database credentials – never committed to Git.                                                                                      |
| DB_NAME                       | Backend  | <code>society_management</code>                                                                                                     |
| JWT_SECRET,<br>JWT_EXPIRES_IN | Backend  | Signing key and lifetime of login tokens (e.g. 8h).                                                                                 |
| CORS_ORIGIN                   | Backend  | Allowed frontend origin, e.g. the Vercel URL.                                                                                       |

In development these values live in `.env.local` (frontend) and `.env` (backend) files that are listed in `.gitignore`. A committed `.env.example` file lists the variable names without values so that new developers know what to provide. In production they are entered in the hosting dashboard's environment settings.

The project's own configuration files are:

- `package.json` – dependencies and the dev, build, lint, and preview scripts; "type": "module" enables ES modules.
- `vite.config.js` – registers the React plugin, which enables JSX and fast refresh.
- `eslint.config.js` – the flat ESLint configuration with React Hooks and React Refresh rules.
- `index.html` – the single HTML entry page with the `#root` element and favicon.

### 9.3 Hosting of Frontend and Backend

**Frontend hosting.** The frontend is hosted as static files on a CDN-backed platform. Static hosting is ideal for this application because the server only has to deliver three files; all rendering happens in the user's browser. Vercel and Netlify both offer a free tier that comfortably covers a society portal, automatic HTTPS certificates, global edge caching, and Git-based deployments. Hashed filenames (for example `index-BG1OJBxa.js`) allow the CDN to cache assets for a long time while guaranteeing that users receive the new version after each deployment.

**Backend hosting (planned).** The Express API will run on a platform-as-a-service such as Render or Railway, which builds the Node.js application from GitHub and restarts it automatically if it stops. For larger societies, an AWS EC2 instance with PM2 as the process manager and Nginx as a reverse proxy is an alternative.

**Database hosting (planned).** The MySQL database will be hosted on a managed service—AWS RDS, Railway MySQL, or Aiven—which provides automated daily backups, encryption at rest, and monitoring. The database will accept connections only from the API server, not from the public internet.

| Component   | Recommended host                        | Notes                                                     |
|-------------|-----------------------------------------|-----------------------------------------------------------|
| Frontend    | Vercel (alternative: Netlify)           | Free tier, HTTPS, CDN, preview deployments, SPA rewrite.  |
| Backend API | Render / Railway (alternative: AWS EC2) | Node.js 20 runtime, environment variables, health checks. |
| Database    | AWS RDS MySQL / Railway MySQL           | MySQL 8, automated backups, private networking.           |

|        |               |                                                                                           |
|--------|---------------|-------------------------------------------------------------------------------------------|
| Domain | Any registrar | e.g. portal.<society>.in pointing to the frontend<br>and api.<society>.in to the backend. |
|--------|---------------|-------------------------------------------------------------------------------------------|

## CHAPTER -10 CHALLENGES & LIMITATIONS

### 10.1 Issues Faced During Development

#### ***1. Modelling ten related modules without repeating code***

The portal has ten modules, and eight of them follow the same pattern: a list, a form, statistics, and filters. Writing eight separate pages would have produced several thousand lines of nearly identical code that would be hard to keep consistent. Every change—such as adding pagination—would have had to be repeated eight times.

#### ***2. Displaying relationships instead of raw IDs***

Because the data follows the relational schema, a payment row stores `bill_id: 2` and `resident_id: 2`, and a visitor row stores `flat_id: 4`. Showing these numbers would be meaningless to users, and typing them into forms would be error-prone. The team needed a way to show "Ananya Rao" and "A-202" while still storing the IDs.

#### ***3. Designing permissions for eight roles***

Eight roles across eleven pages and four kinds of action (view, create, edit, delete) produce hundreds of combinations. Scattering checks such as `if (user.role === 'Admin')` through many components quickly becomes inconsistent—for example, a button could be hidden on one page while its URL could still be opened directly.

#### ***4. Keeping residents' data private***

Residents must use the same pages as administrators (Maintenance, Payments, Complaints, Visitors, Bookings) but must see only their own records. In addition, when a resident creates a complaint or a visitor entry, they should not be able to attach it to someone else's account or flat.

#### ***5. Keeping all pages in sync after a change***

When a complaint is resolved on the Complaints page, the admin dashboard chart, the resident's dashboard, and the maintenance staff queue should all reflect it. Fetching data separately in each page would lead to stale numbers.

#### ***6. Type mismatches between form inputs and data***

HTML inputs and selects always return strings, while IDs and amounts in the data are numbers. Without care, comparisons such as `row.flat_id === value` fail silently after editing, and amounts are joined as text instead of added (`"5000" + "6500" = "50006500"`).

## 7. Responsive layout for dense tables and dashboards

The admin dashboard has six cards and five panels, and some tables have eight columns. On a phone these layouts overflowed the screen, and the fixed sidebar consumed most of the width.

## 8. Building and testing without a running backend

The backend was not ready while the interface was being built, but the team still needed realistic data, loading states, and asynchronous behaviour to develop and test against.

## 10.2 Solutions Applied

| # | Issue                         | Solution                                                                                                                                                                                                                                                 |
|---|-------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1 | Repeated module code          | Created a declarative resources.js configuration and a single generic ResourcePage, DataTable, and RecordForm. Each module page is now two lines long, and features like sorting and pagination were written once.                                       |
| 2 | Raw IDs in tables and forms   | Added relationship column types (flat, resident, amenity) that look up the related record for display, and relationship field types that render dropdowns populated from the related table.                                                              |
| 3 | Scattered permission checks   | Moved all rules into permissions.js with small, named functions (canAccessPath, canCreateResource, canEditRecord, canDeleteRecord, scopeRecords). The sidebar, route guard, global search, and buttons all use the same functions, so they always agree. |
| 4 | Resident privacy              | scopeRecords filters rows by the resident's flat_id or resident_id; for residents the ownership fields are removed from forms and filled automatically from the session before saving.                                                                   |
| 5 | Stale data across pages       | Introduced a single AppContext that loads all resources once and performs all mutations, so every page reads the same state and re-renders after any change.                                                                                             |
| 6 | String/number mismatches      | Normalized values before saving (fields ending in _id, amount, and floor_number become numbers) and compared IDs with Number() on both sides throughout the code.                                                                                        |
| 7 | Mobile layout                 | Used CSS Grid with breakpoints, horizontally scrollable table containers, and an off-canvas sidebar with an overlay that closes on tap or navigation.                                                                                                    |
| 8 | No backend during development | Built an asynchronous service layer with simulated latency and sample data that mirrors the MySQL columns, so the switch to a real API changes only one file.                                                                                            |

## 10.3 Current Limitations or Known Bugs

The following limitations and defects were identified through code review and the test results in Chapter 8. They are recorded honestly so that they can be addressed in the next phase.

### **Limitations of the current release**

1. **No persistent storage.** Data lives in memory; changes are lost when the page is reloaded (test case TC-23). The user's session, however, remains in localStorage, so a user stays signed in while their edits disappear.
2. **Frontend-only security.** Authentication uses demo accounts and permission checks run in the browser. Passwords in the sample data are stored as plain text. This is acceptable for a demonstration but must be replaced by server-side authentication with hashed passwords before real use.
3. **No real payment processing.** Payments are recorded manually; there is no gateway integration, and recording a successful payment does not automatically mark the related bill as Paid.
4. **No booking conflict checks.** The system does not yet prevent an end time earlier than the start time, overlapping bookings of the same amenity, or bookings for an amenity under maintenance (the Book button is disabled, but the bookings form still lists it).
5. **Fixed demonstration date and sample notifications.** "Today" is fixed to 8 September 2026 in some statistics and greetings to match the sample data, and the notification panel and activity feed show sample entries.
6. **Settings are not saved.** The profile form confirms the save but does not yet update the stored user.

### **Known bugs**

| # | Bug                                       | Details and planned fix                                                                                                                                                                                                         |
|---|-------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1 | Incorrect success message wording (TC-09) | The toast is built by removing the last letter of the resource name, producing "Staf added successfully." and "Maintenanc added successfully." Fix: use the itemName from the resource configuration.                           |
| 2 | Password included in table search         | Search checks every field of a resident record, including the password, and the edit form shows the existing password. Fix: exclude sensitive fields from search and leave the password field blank on edit.                    |
| 3 | Residents can set status fields           | A resident's booking and complaint forms still include the status dropdown, so a resident could mark their own booking Confirmed or complaint Resolved. Fix: hide status fields for residents and default them to Pending/Open. |
| 4 | Mislabelled statistics                    | "Visitors Today" on the Visitors page counts all visitors; "Active residents" repeats the total; "6 flats pending" on the dashboard actually counts bills.                                                                      |

|   |                                 |                                                                                                                                                                                                                         |
|---|---------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 5 | Phone validation message        | For an invalid phone number the browser's native message ("Please match the requested format.") appears instead of the custom message, because the input's pattern attribute triggers browser validation first (TC-05). |
| 6 | Pagination not reset by filters | After filtering, the stored page number is not reset to 1; the display is corrected automatically but the state is stale.                                                                                               |
| 7 | No error handling on save       | Failed service calls are not caught and there is no guard against double submission. This will matter once real network calls are introduced.                                                                           |
| 8 | Large single bundle             | The JavaScript bundle is 686 KB (202 KB gzipped) because Recharts is loaded on first visit; route-level code splitting would reduce initial load time.                                                                  |
| 9 | Accessibility gaps              | Dialogs do not close with the Escape key or trap keyboard focus, and clickable table rows are not reachable with the keyboard.                                                                                          |

## CHAPTER -11 FUTURE ENHANCEMENTS

### 11.1 Planned Features

#### ***1. Production backend with MySQL persistence***

The highest-priority enhancement is the Node.js and Express REST API connected to the society\_management MySQL database. The service layer has been designed so that only api.js needs to change. The backend will implement every endpoint listed in Section 8.1, enforce all permission rules in middleware, and use database transactions where several tables change together.

#### ***2. Secure authentication***

Real user accounts will replace demo accounts. Passwords will be hashed with bcrypt, logins will return a signed JSON Web Token with an expiry time, and the frontend will attach the token to every request. Additional features will include password reset by email or OTP, account lockout after repeated failed attempts, and an option for administrators to invite new residents by email.

#### ***3. Online maintenance payments***

Integration with a UPI-enabled payment gateway such as Razorpay will let residents pay bills directly from the portal. A successful payment webhook will create the payment record and mark the bill Paid in the same transaction. Residents will be able to download a PDF receipt, and the treasurer will be able to export monthly collection reports to Excel.

#### ***4. Automatic billing and reminders***

A scheduled job will generate each month's bills automatically from a rate table (for example by flat type or area), mark unpaid bills as Overdue after the due date, and apply late fees according to society rules. Residents will receive reminders before and after the due date.

#### ***5. Notifications***

The notification bell will show real, per-user events: new bills, payment confirmations, complaint status changes, visitor arrivals, and new notices. Notifications will also be delivered by email and SMS, and later as push notifications.

#### ***6. Smarter visitor management***

Residents will be able to generate a one-time QR or numeric gate pass for an expected guest. The security guard will scan or enter the pass to check the visitor in, and the resident will be notified instantly. Frequent visitors such as domestic help can receive recurring passes, and visitor photographs can be captured at the gate.

## 7. Amenity booking rules and calendar

Bookings will be validated against amenity opening hours, maximum duration, and existing reservations to prevent conflicts. A calendar view will show each amenity's availability by day and week, and the committee will be able to block slots for maintenance or society events.

## 8. Complaint assignment and SLAs

Complaints will be assigned to specific staff members, categorized (plumbing, electrical, lift, security), and tracked against target resolution times. Residents will be able to attach photos and rate the resolution, and the committee will see average resolution time per category.

## 9. Community features

Planned community features include polls and voting for committee decisions, an event calendar, a document repository for bylaws and meeting minutes, and a resident directory with privacy controls.

## 10. Mobile application

A mobile app built with React Native, sharing the same API and much of the same logic, will provide quicker access for residents and guards, with push notifications and camera access for QR passes and complaint photos.

## 11.2 Possible Integrations or Optimizations

### Integrations

| Integration                                         | Benefit                                                                  |
|-----------------------------------------------------|--------------------------------------------------------------------------|
| Razorpay / UPI gateway                              | Online payment of maintenance bills with automatic reconciliation.       |
| Email (SMTP / SendGrid)<br>and SMS (MSG91 / Twilio) | Bill reminders, OTP login, visitor alerts, and notice broadcasts.        |
| WhatsApp Business API                               | Send notices and bill reminders on the channel residents already use.    |
| Google Calendar                                     | Add amenity bookings and society events to residents' calendars.         |
| Cloud storage (AWS S3 /<br>Cloudinary)              | Store complaint photos, visitor photos, receipts, and society documents. |
| Accounting export (Tally /<br>Excel)                | Export collections and expenses for the society's auditor.               |
|                                                     | Connect QR passes to boom barriers or door controllers.                  |

Smart gate hardware

### Optimizations

- **Code splitting.** Load each route with React.lazy and Suspense so that the Recharts library is downloaded only when the admin dashboard is opened, reducing the initial bundle.

- **Server-side pagination and filtering.** As data grows, request one page of records at a time with query parameters such as `?page=2&status=Pending` instead of loading every record at start-up.
- **Data caching.** Use a data-fetching library such as TanStack Query for caching, background refresh, and optimistic updates.
- **Efficient session handling.** Read the current user once into context instead of parsing `localStorage` on every render, so memoized calculations are not recomputed unnecessarily.
- **Database indexing.** Add indexes on foreign keys and on status and date columns used in filters and dashboards.
- **Automated testing in CI.** Run the Playwright suite and unit tests for the permission functions on every pull request.
- **Accessibility improvements.** Add Escape-to-close and focus trapping in dialogs, keyboard access to table rows, and verify colour contrast against WCAG 2.1 AA.
- **Progressive Web App.** Add a web manifest and service worker so the portal can be installed on phones and show cached notices offline.
- **Multi-society support.** Add a `society_id` to each table so that one installation can serve several societies, which would make the system viable as a hosted service.

## CHAPTER -12 CONCLUSION

### 12.1 Summary of the Project

The Society/Apartment Management Portal—Haven Society Management—was developed to replace the scattered registers, spreadsheets, and chat messages that many residential societies rely on with a single, organized, role-aware web application. The project began from a clear problem: society information about flats, people, money, communication, and facilities is spread across many places and many people, and as a result it is duplicated, inconsistent, and difficult to act on.

To address this problem, the team first modelled the domain as a relational database named `society_management` with ten normalized tables—flats, residents, visitors, staff, maintenance\_bills, payments, complaints, notices, amenities, and amenity\_bookings—connected by eight one-to-many relationships. On top of this model, a complete frontend was built with React 19 and Vite 8, using React Router for navigation, Recharts for analytics, and Lucide for icons.

The application provides ten functional modules, an analytics dashboard for administrators, tailored dashboards for seven other roles, and full create, read, update, and delete operations with search, filters, sorting, pagination, and validation. A central permission configuration controls which pages, actions, and records each of the eight roles can access, and record-level scoping ensures that residents see only their own information. All data access passes through a single asynchronous service layer, which currently uses sample data that mirrors the database columns and is designed to be replaced by a REST API with minimal change.

The project was verified through static analysis (ESLint, zero problems), a successful production build, and 23 automated browser test cases across all roles, of which 21 passed. The two failing cases—a wording defect and the expected loss of in-memory data on reload—are documented along with other known limitations and a clear plan for the next phase.

### 12.2 What Was Achieved

- **A working, responsive portal** covering all ten areas of society operations, usable on desktops, tablets, and phones.
- **A normalized ten-table relational design** with clearly defined primary keys, foreign keys, enumerated statuses, and an entity-relationship diagram, ready to be created in MySQL.
- **Role-based access for eight roles** at three levels—route, action, and record—defined in one maintainable configuration file and applied consistently across the sidebar, route guard, global search, and buttons.

- **A configuration-driven architecture** in which eight modules share one generic page, table, and form, significantly reducing code size and making new modules easy to add.
- **Meaningful dashboards:** an admin dashboard with six KPIs and three charts, and role dashboards with quick actions for residents, finance, communications, committee, security, housekeeping, and maintenance staff.
- **Data quality at entry** through relationship dropdowns and validation for required fields, email, and phone numbers.
- **Backend-ready design:** an asynchronous service layer, a REST endpoint map aligned with the database tables, a Postman test plan, and a deployment architecture for frontend, API, and database.
- **Verified quality:** lint-clean code, a successful production build, and a documented automated test suite with real results.

Compared with the manual process described in the problem statement, the portal allows an administrator to see occupancy, dues, open complaints, and visitors on one screen; allows a treasurer to link every payment to a bill and a resident; allows a resident to check dues and raise a complaint without calling the office; and allows the security guard to maintain a searchable visitor log instead of a notebook.

## 12.3 Skills Learned During Development

### *Technical skills*

- **React development:** building function components, managing state with `useState`, handling side effects with `useEffect`, optimizing with `useMemo`, and sharing global state with the Context API and a custom hook.
- **Client-side routing:** nested routes, layout routes with `Outlet`, redirects, and route protection with React Router.
- **Configuration-driven design:** describing modules as data and writing generic components that interpret that data.
- **Relational database design:** identifying entities and relationships, choosing keys and data types, normalizing to third normal form, and drawing ER diagrams.
- **Access control:** designing role-based and record-level permission rules, and understanding why client-side checks must be repeated on the server.
- **Asynchronous JavaScript:** Promises, `async/await`, and designing a service layer that can switch from mock data to HTTP without changing its callers.
- **Data visualization:** preparing data and building responsive charts with Recharts.

- **Responsive CSS:** CSS Grid, Flexbox, media queries, and off-canvas navigation.
- **Tooling and quality:** Vite builds, ESLint configuration, Git workflows, browser-based automated testing with Playwright, and API test planning with Postman.
- **Deployment:** building static assets, configuring SPA rewrites, environment variables, and planning cloud hosting for API and database.

### *Professional skills*

- **Requirement analysis:** translating the everyday work of committees, residents, and staff into roles, modules, and permissions.
- **Teamwork and version control:** dividing work among team members and integrating it through a shared repository.
- **Honest evaluation:** testing against real behaviour, recording failures as well as successes, and documenting limitations clearly.
- **Technical documentation:** writing structured reports with diagrams, schema definitions, test cases, and setup instructions.

In conclusion, the Society Management Portal demonstrates that a small team can build a structured, maintainable, and user-friendly system for a real-world problem using modern web technologies and sound database principles. The foundation laid in this phase—the data model, the role model, the reusable interface, and the service boundary—positions the project well for the next step of connecting a secure backend and putting the portal into use in a real residential society.

## CHAPTER -13 REFERENCES

### Books

Pressman, R. S., & Maxim, B. R. (2019). *Software Engineering: A Practitioner's Approach* (9th ed.).

McGraw-Hill Education.

Sommerville, I. (2016). *Software Engineering* (10th ed.). Pearson Education.

Elmasri, R., & Navathe, S. B. (2016). *Fundamentals of Database Systems* (7th ed.). Pearson.

Silberschatz, A., Korth, H. F., & Sudarshan, S. (2019). *Database System Concepts* (7th ed.).

McGraw-Hill Education.

Duckett, J. (2014). *HTML & CSS: Design and Build Websites*. Wiley.

Haverbeke, M. (2018). *Eloquent JavaScript* (3rd ed.). No Starch Press.

Krug, S. (2014). *Don't Make Me Think, Revisited: A Common Sense Approach to Web Usability* (3rd ed.). New Riders.

### Official Documentation

React. (n.d.). *React documentation – Learn React and API Reference*. Retrieved from

<https://react.dev>

Vite. (n.d.). *Vite Guide and Configuration Reference*. Retrieved from <https://vite.dev>

React Router. (n.d.). *React Router Documentation*. Retrieved from <https://reactrouter.com>

Recharts. (n.d.). *Recharts API and Examples*. Retrieved from <https://recharts.org>

Lucide. (n.d.). *Lucide Icons – React package*. Retrieved from <https://lucide.dev>

ESLint. (n.d.). *ESLint User Guide*. Retrieved from <https://eslint.org/docs>

Oracle. (n.d.). *MySQL 8.0 Reference Manual*. Retrieved from

<https://dev.mysql.com/doc/refman/8.0/en/>

OpenJS Foundation. (n.d.). *Node.js Documentation*. Retrieved from <https://nodejs.org/docs>

Express. (n.d.). *Express – Node.js web application framework, API reference*. Retrieved from

<https://expressjs.com>

Microsoft. (n.d.). *Playwright Documentation*. Retrieved from <https://playwright.dev>

Postman. (n.d.). *Postman Learning Center – Writing tests*. Retrieved from

<https://learning.postman.com>

Vercel. (n.d.). *Deploying Vite projects*. Retrieved from <https://vercel.com/docs>

Git. (n.d.). *Pro Git book and reference*. Retrieved from <https://git-scm.com/doc>

## **Tutorials and Learning Resources**

MDN Web Docs. (n.d.). *HTML, CSS, JavaScript, Web Storage API and Intl reference*. Retrieved from <https://developer.mozilla.org>

W3Schools. (n.d.). *React, SQL, and CSS Tutorials*. Retrieved from <https://www.w3schools.com>

GeeksforGeeks. (n.d.). *ER Model, Normalization, and DBMS Tutorials*. Retrieved from <https://www.geeksforgeeks.org>

freeCodeCamp. (n.d.). *Front End Development Libraries and Relational Database curricula*. Retrieved from <https://www.freecodecamp.org>

## **Standards and Security References**

OWASP Foundation. (2021). *OWASP Top 10 Web Application Security Risks*. Retrieved from <https://owasp.org/Top10/>

Jones, M., Bradley, J., & Sakimura, N. (2015). *JSON Web Token (JWT)* (RFC 7519). IETF. Retrieved from <https://www.rfc-editor.org/rfc/rfc7519>

W3C. (2018). *Web Content Accessibility Guidelines (WCAG) 2.1*. Retrieved from <https://www.w3.org/TR/WCAG21/>

Fielding, R. T. (2000). *Architectural Styles and the Design of Network-based Software Architectures* (Doctoral dissertation, University of California, Irvine).

## CHAPTER -14 APPENDICES

### A. Screenshots of the App

The following screenshots were captured from the running application in a Chromium browser at 1440 × 900 resolution.

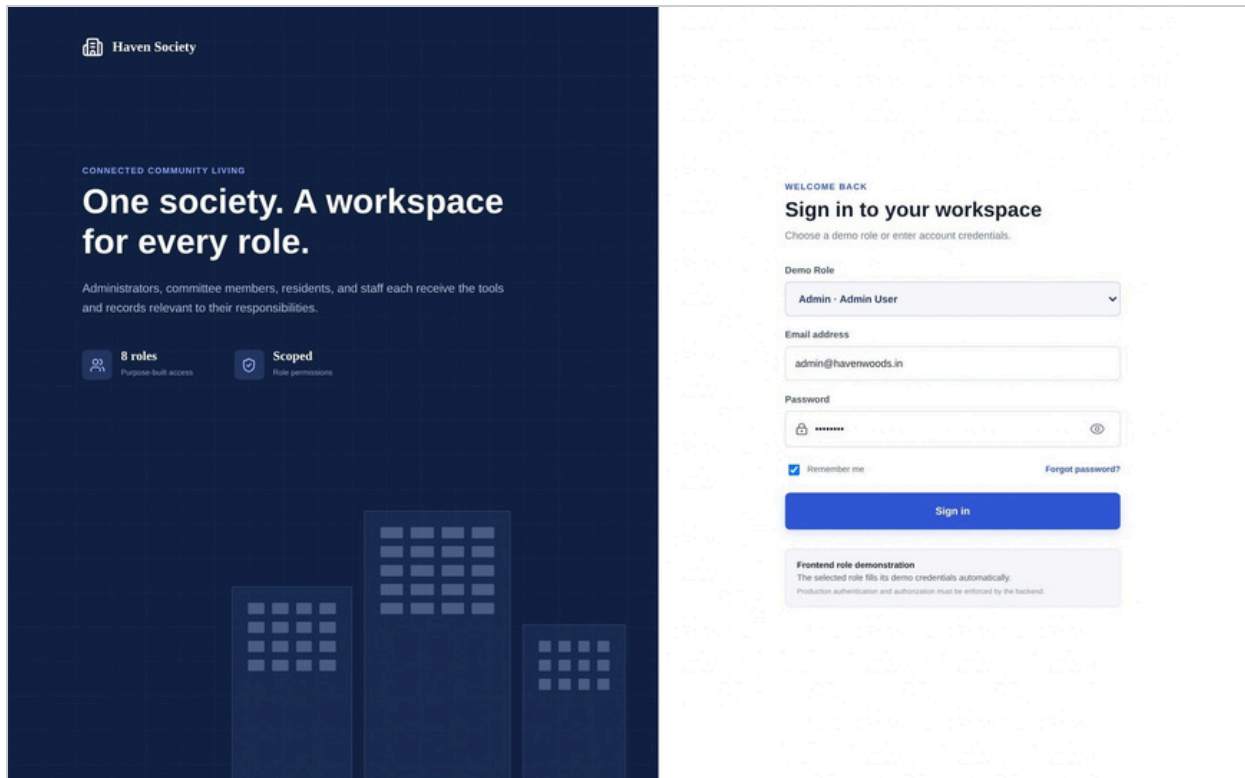


Figure A.1 – Login page with demo role selector

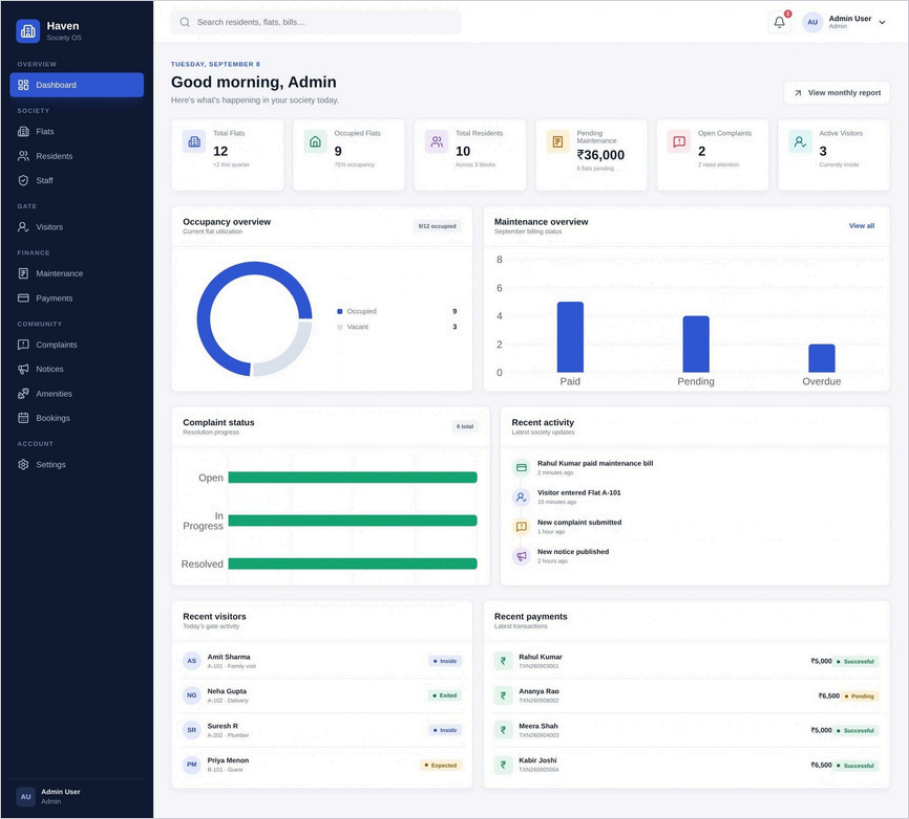


Figure A.2 – Admin dashboard – KPI cards, occupancy, maintenance and complaint charts

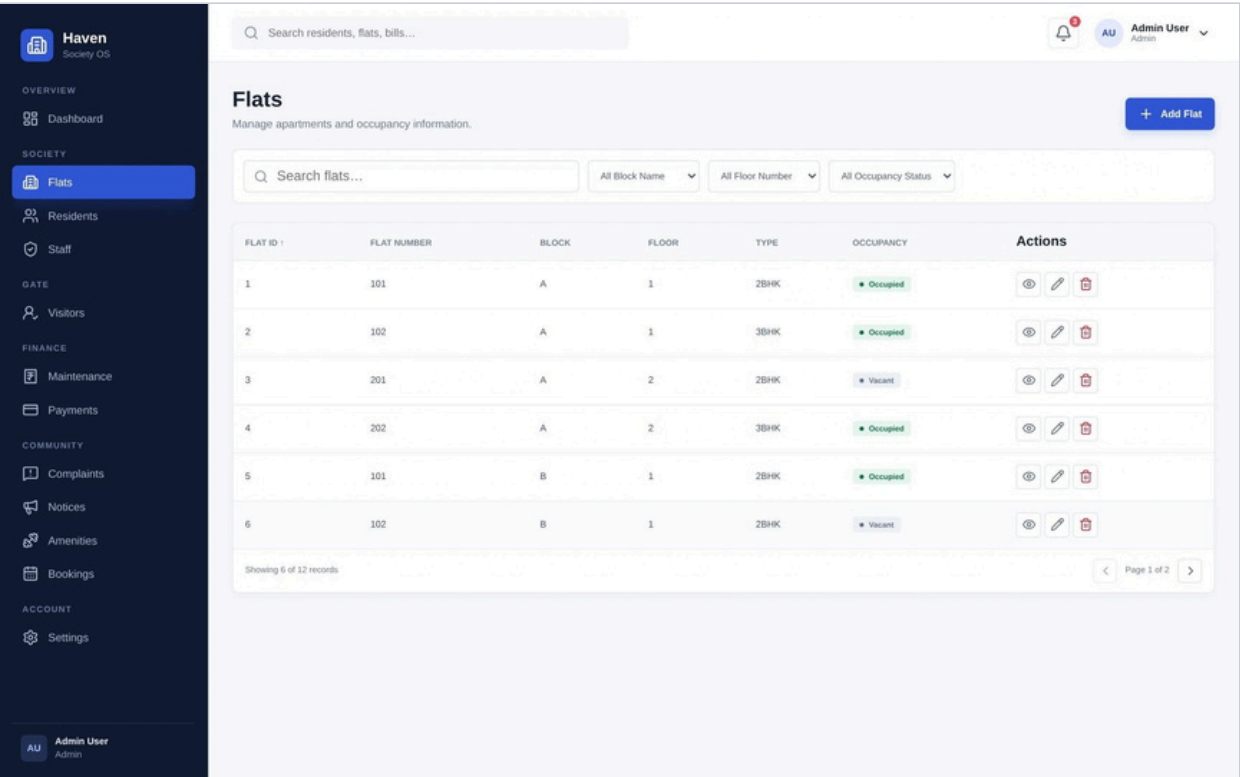


Figure A.3 – Flats module with block, floor and occupancy filters

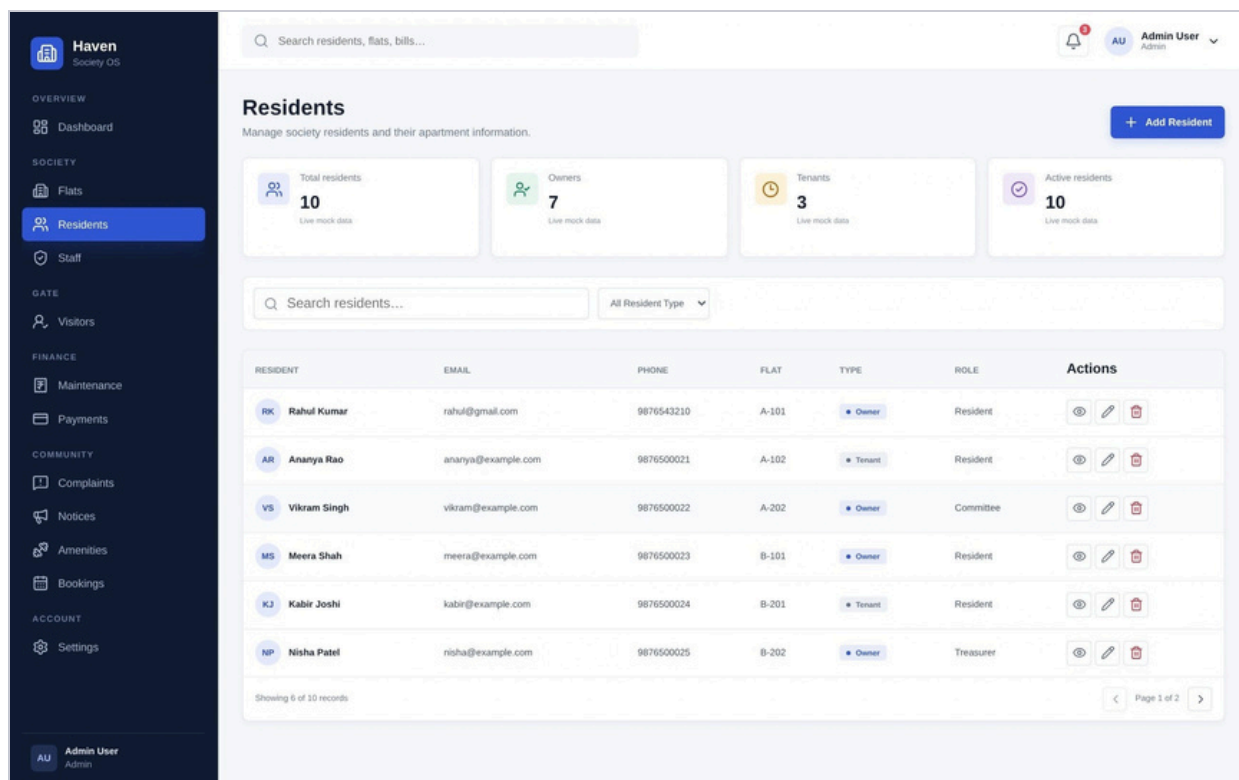


Figure A.4 – Residents module

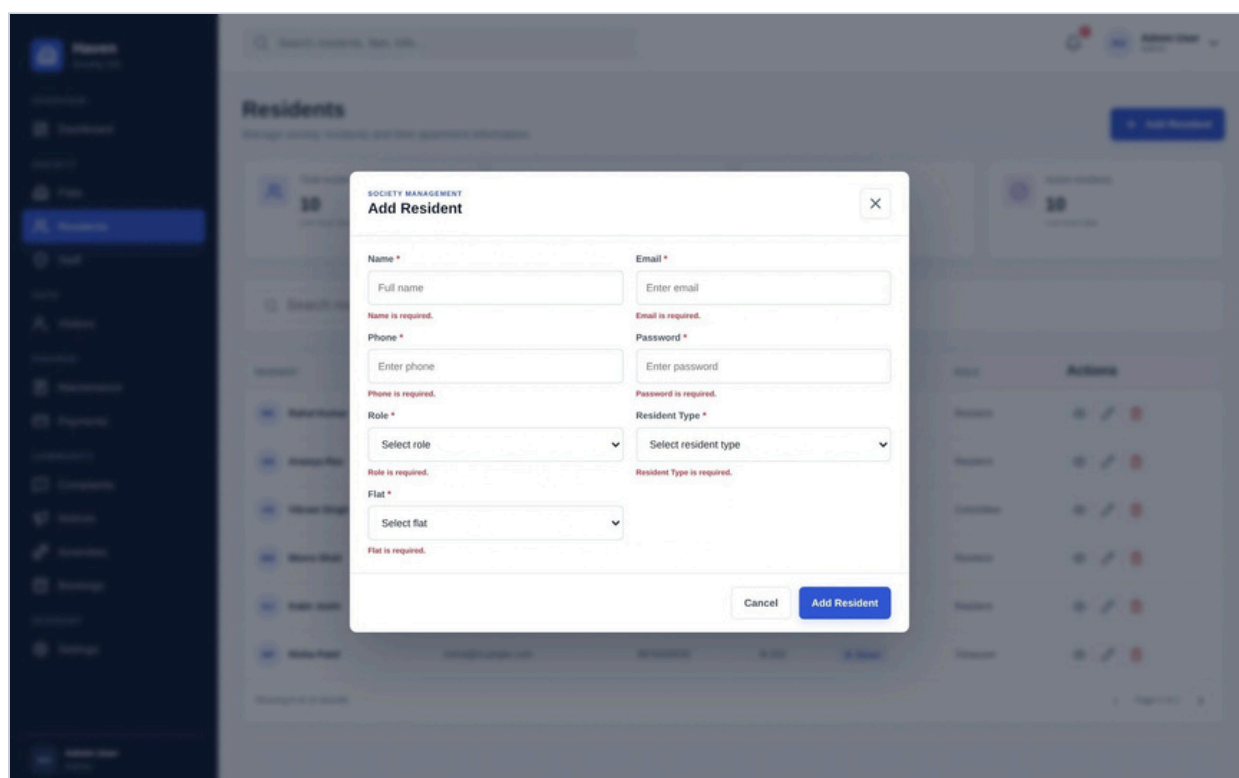


Figure A.5 – Add Resident form showing required-field validation

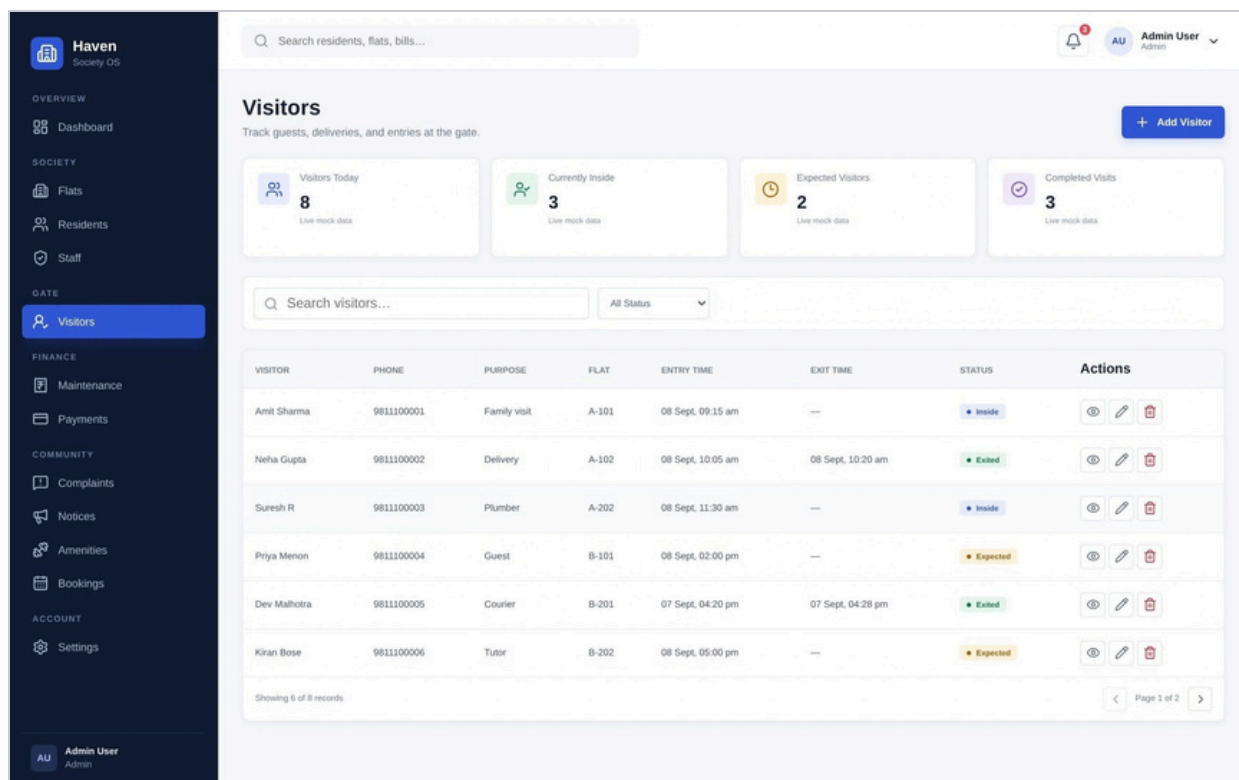


Figure A.6 – Visitors (gate register) module

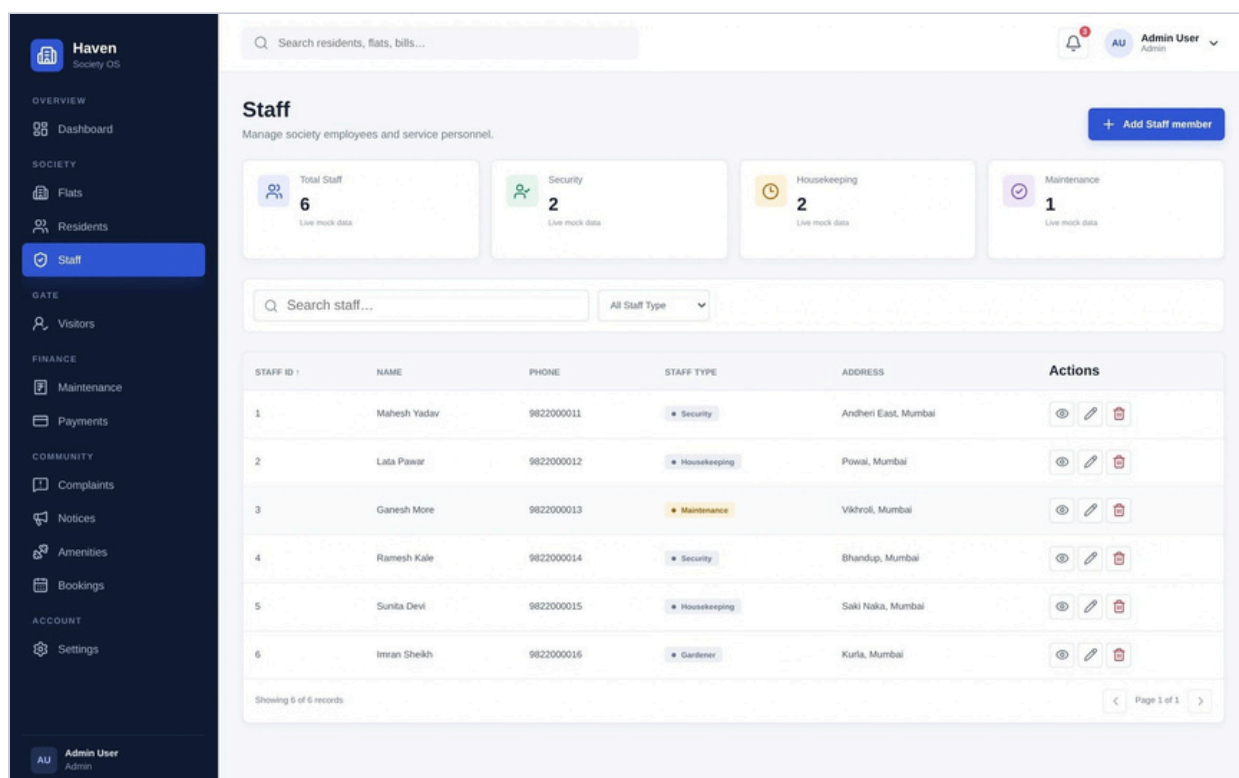


Figure A.7 – Staff module

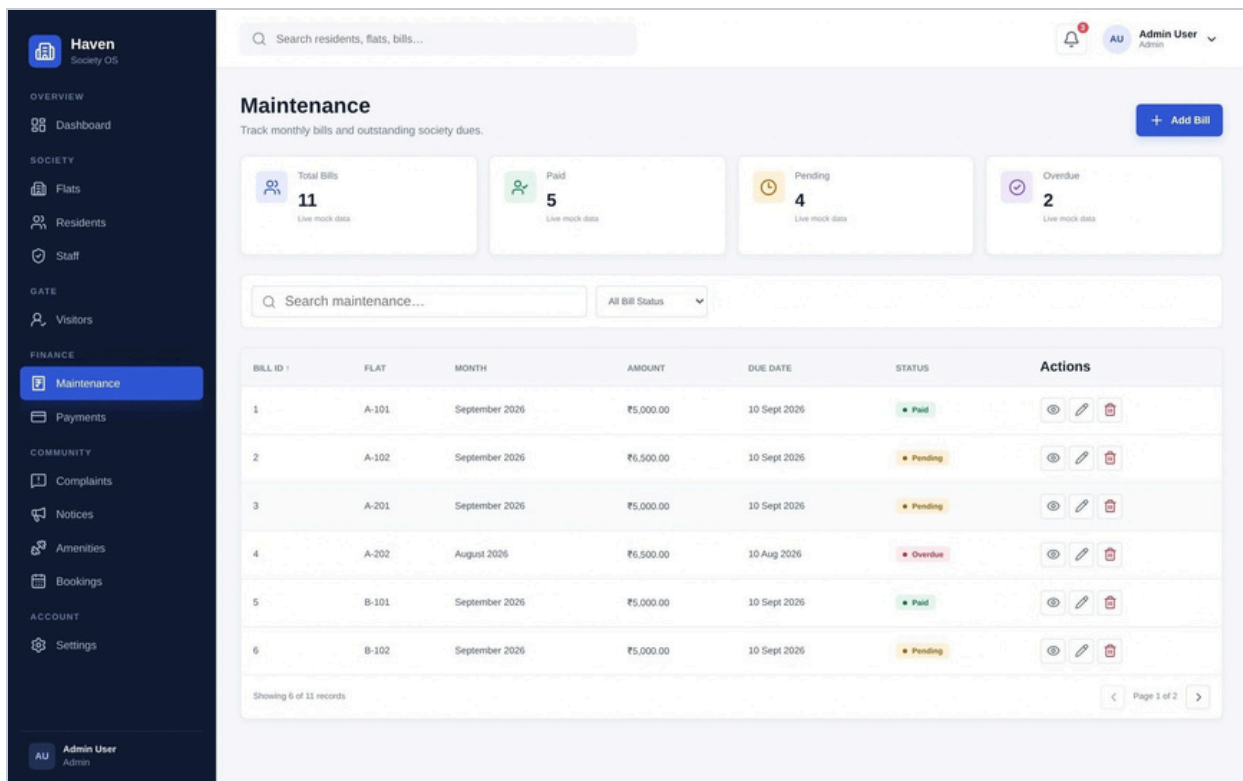


Figure A.8 – Maintenance bills module

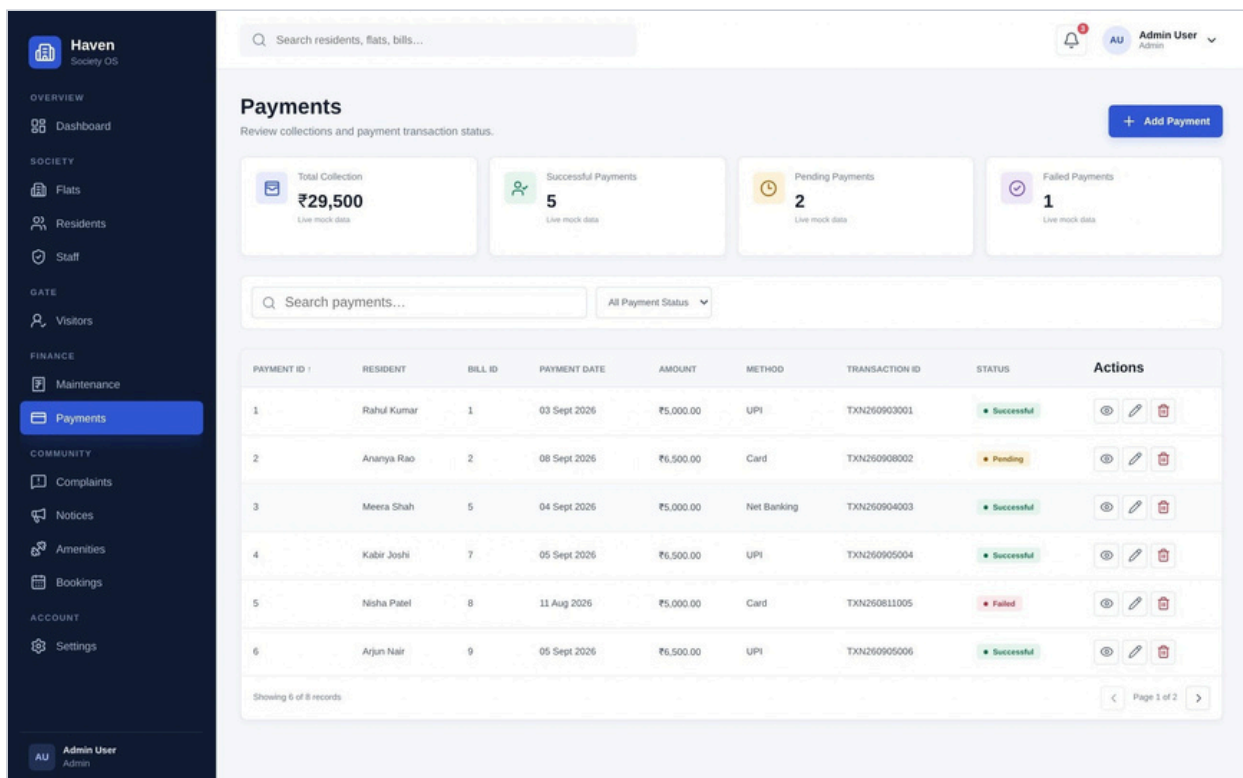


Figure A.9 – Payments module with total collection

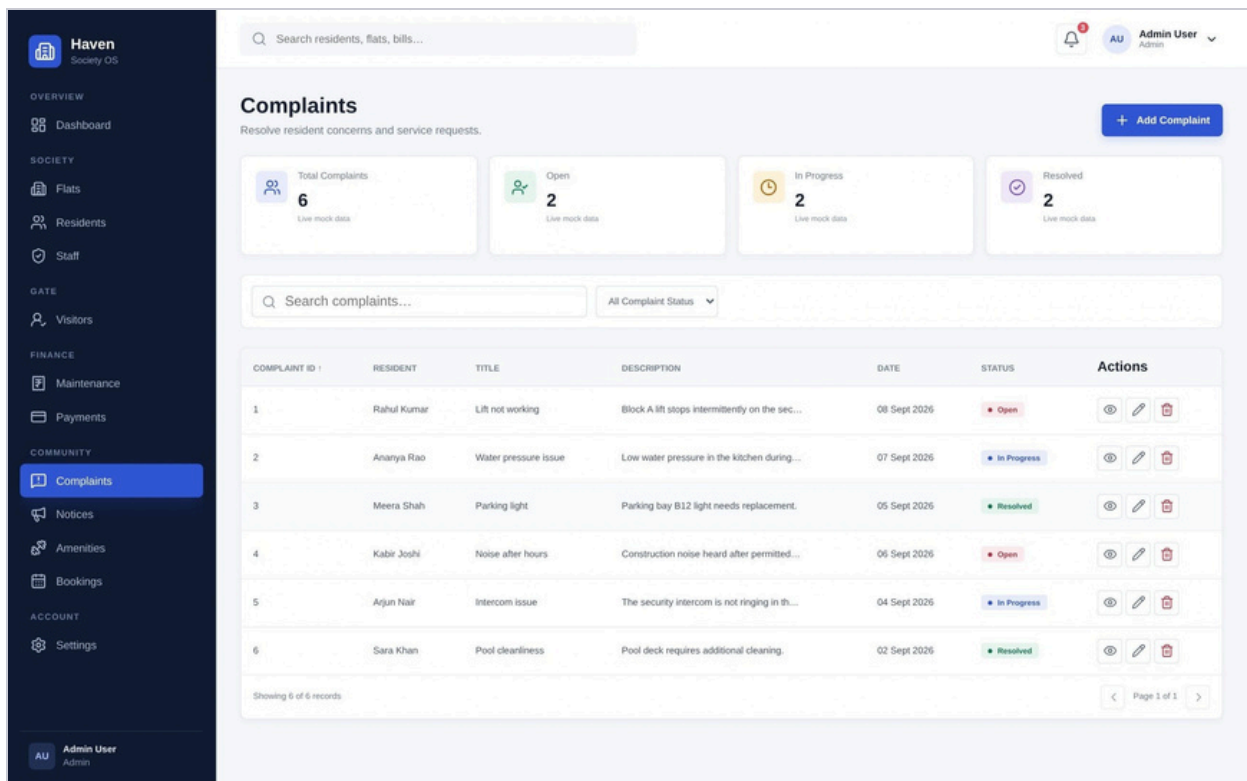


Figure A.10 – Complaints module

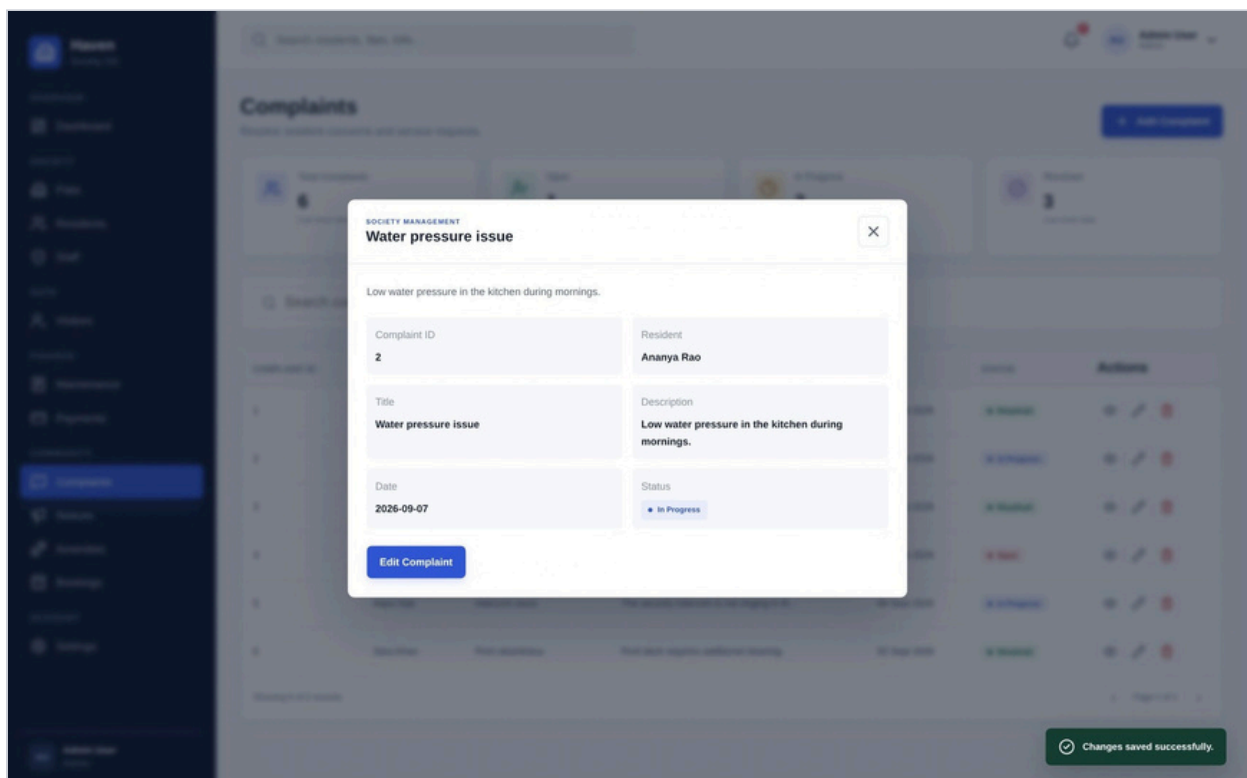


Figure A.11 – Complaint detail dialog

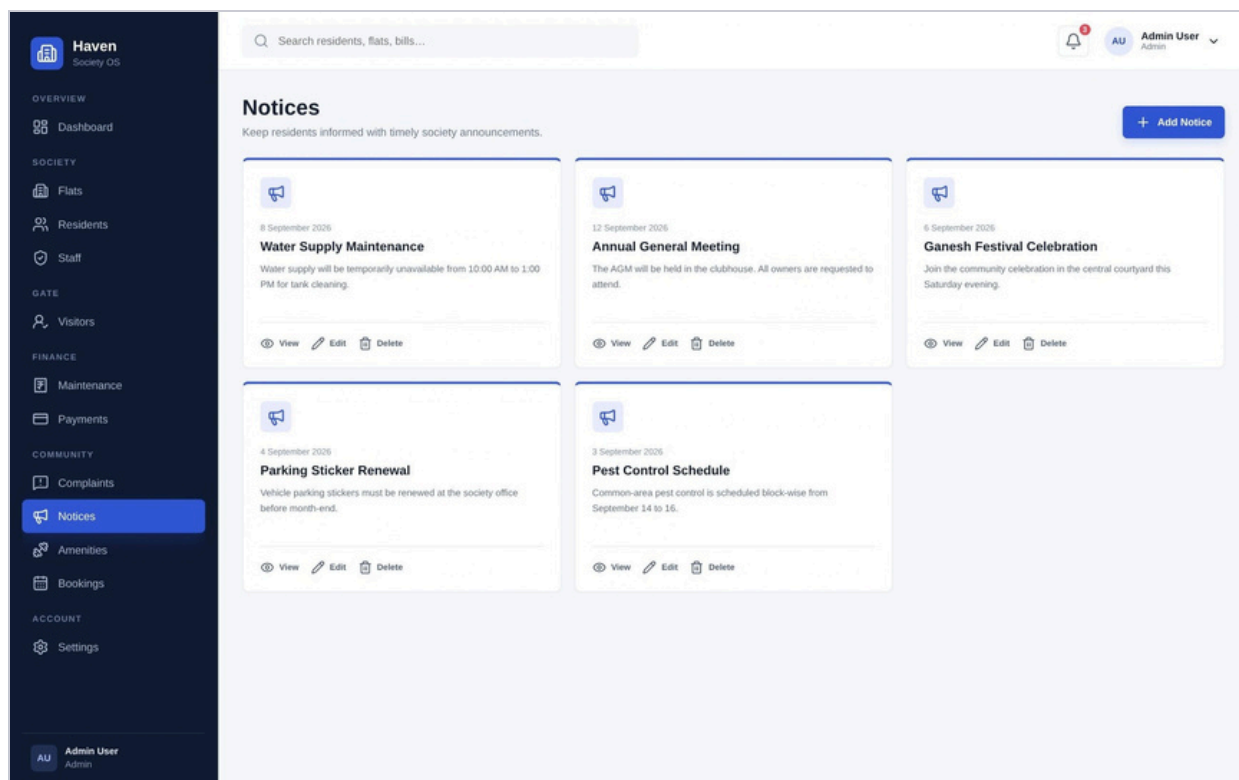


Figure A.12 – Notices board

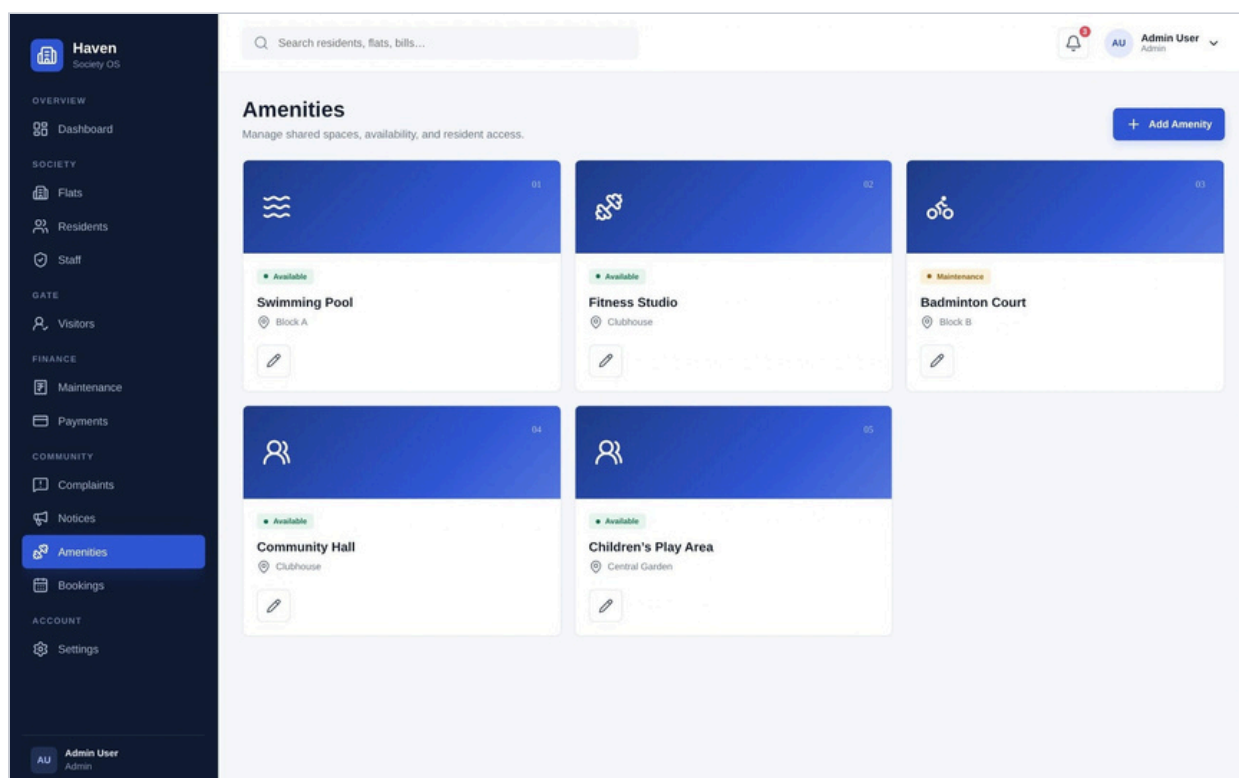


Figure A.13 – Amenities

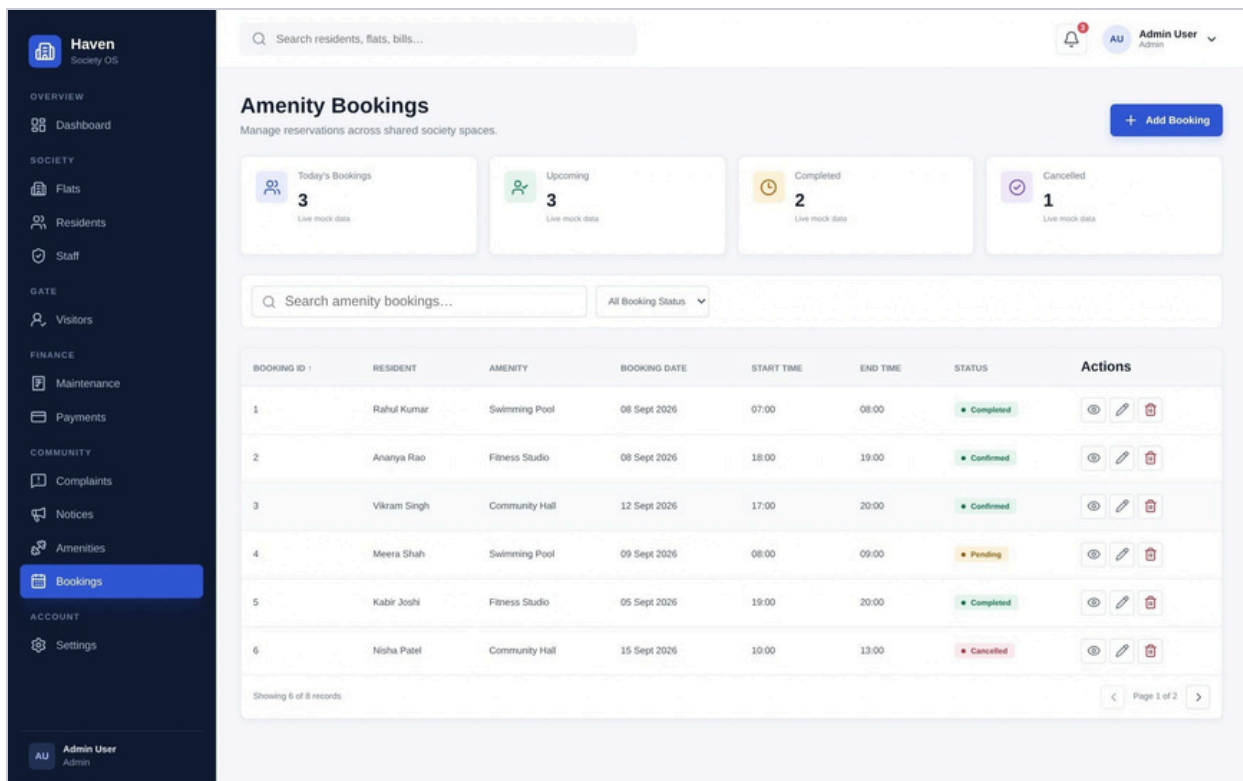


Figure A.14 – Amenity bookings

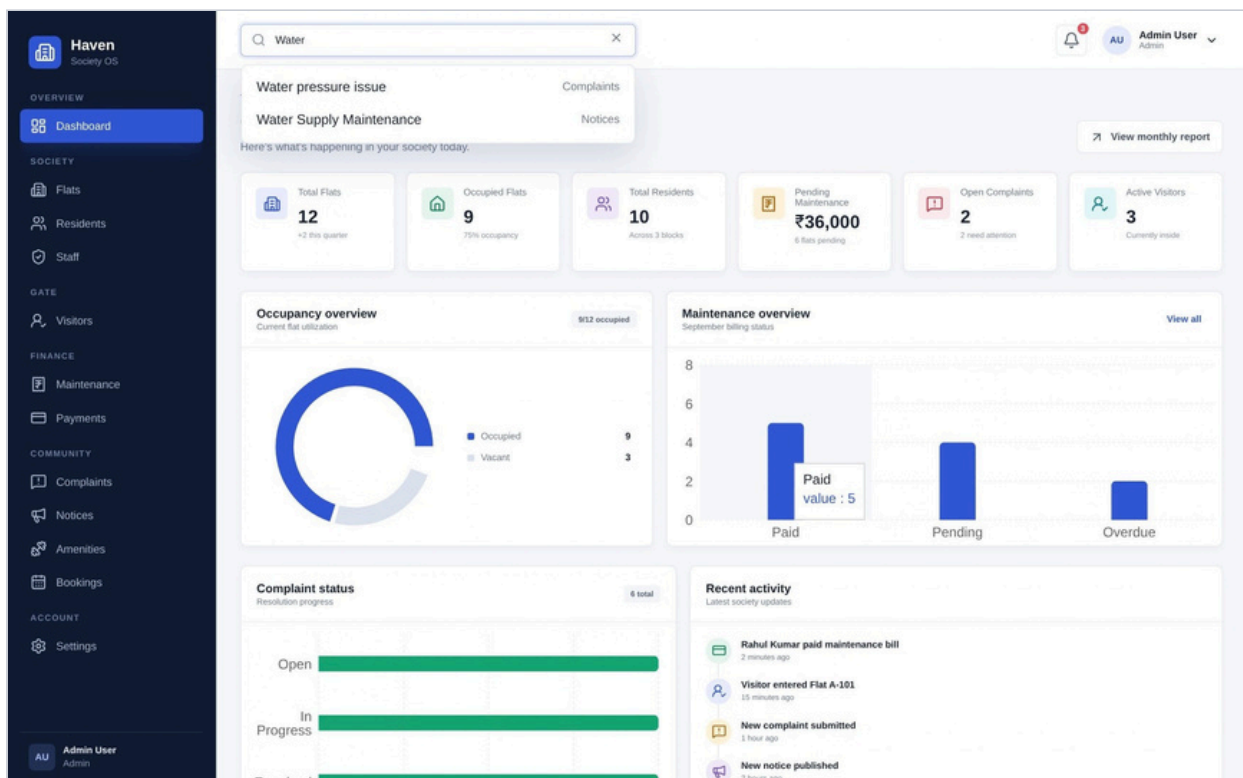


Figure A.15 – Global search results

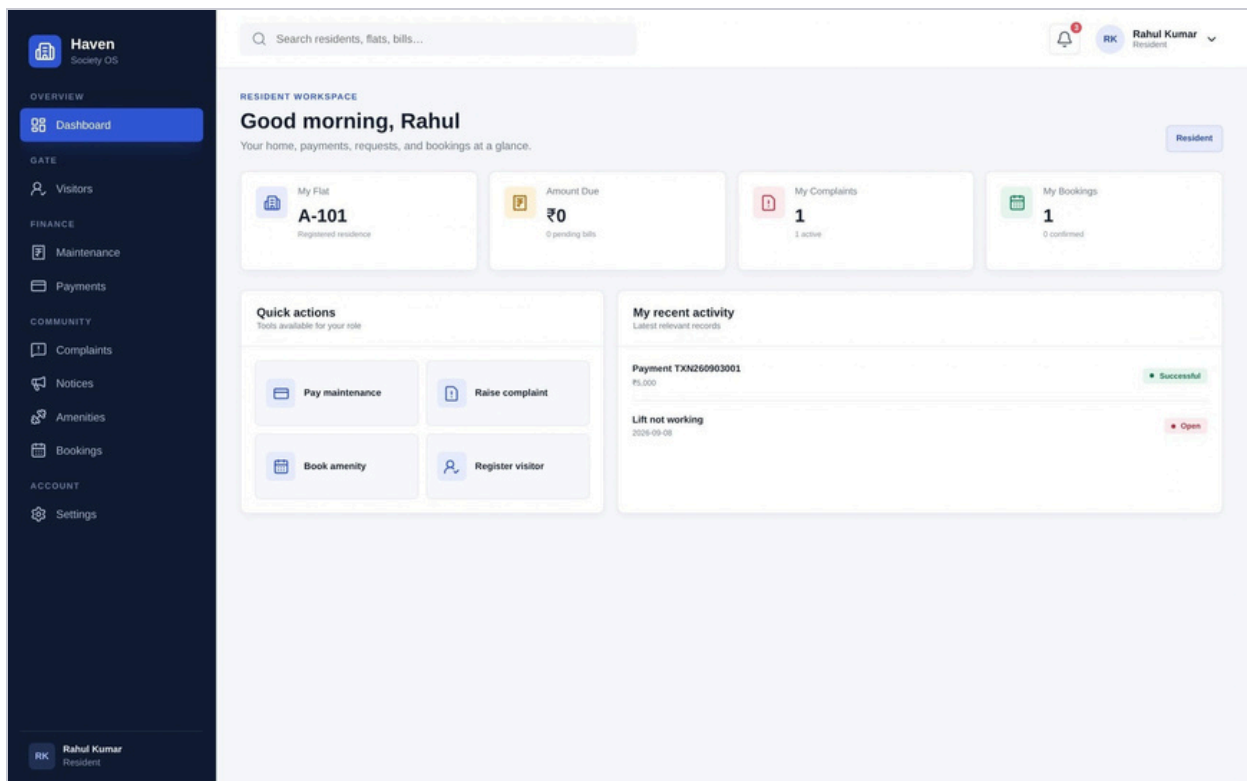


Figure A.16 – Resident dashboard (Rahul Kumar, Flat A-101)

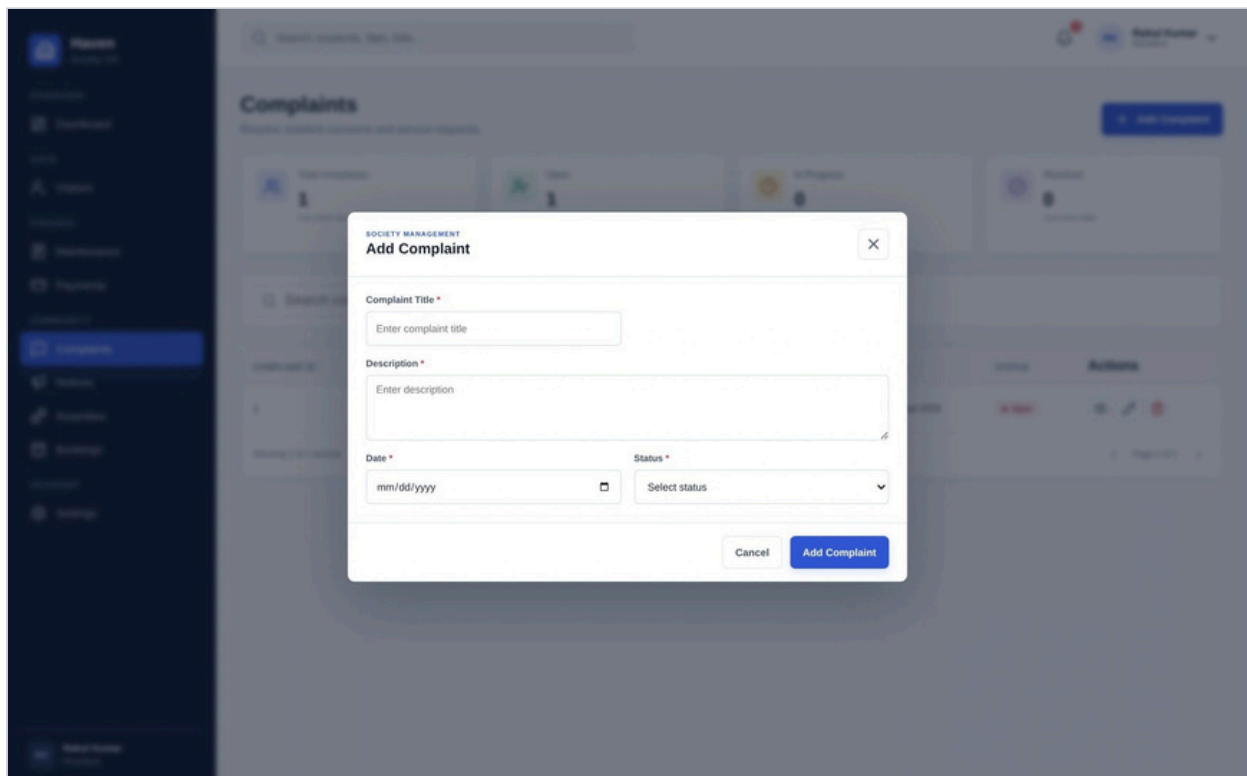


Figure A.17 – Resident raising a complaint – resident field filled automatically

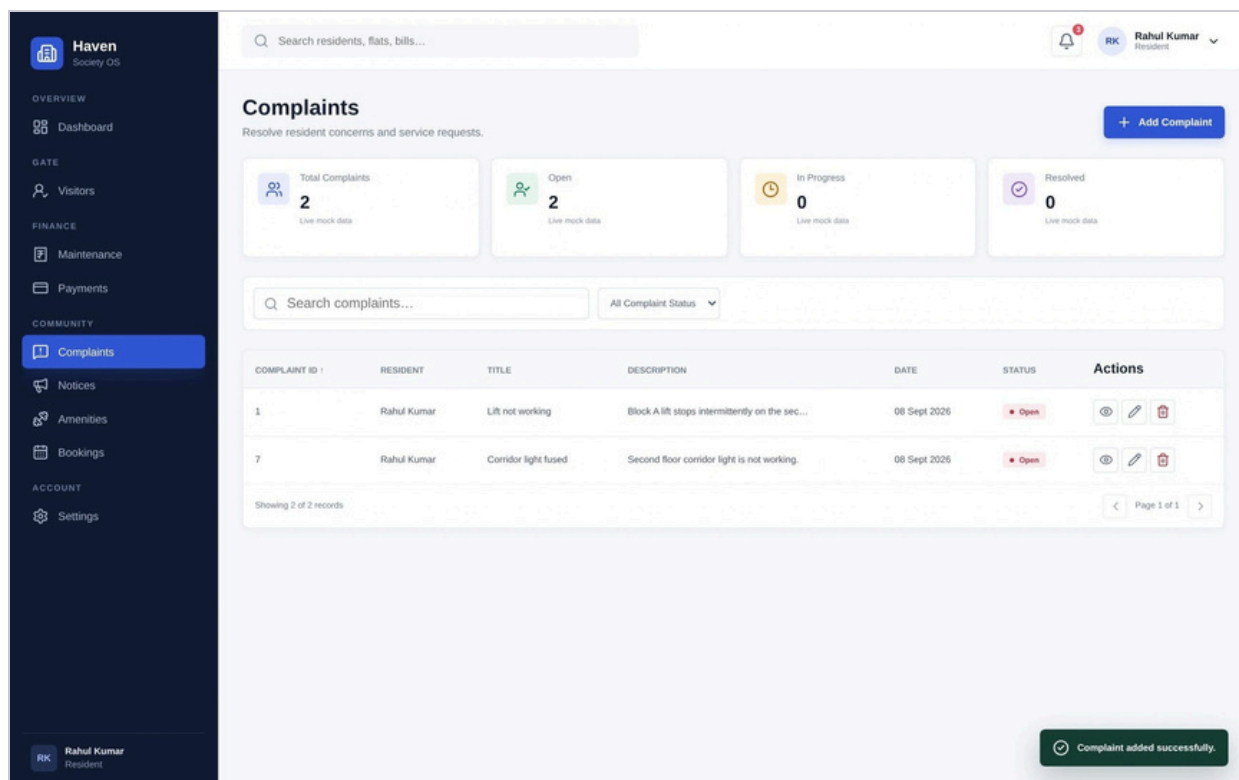


Figure A.18 – Resident sees only their own complaints

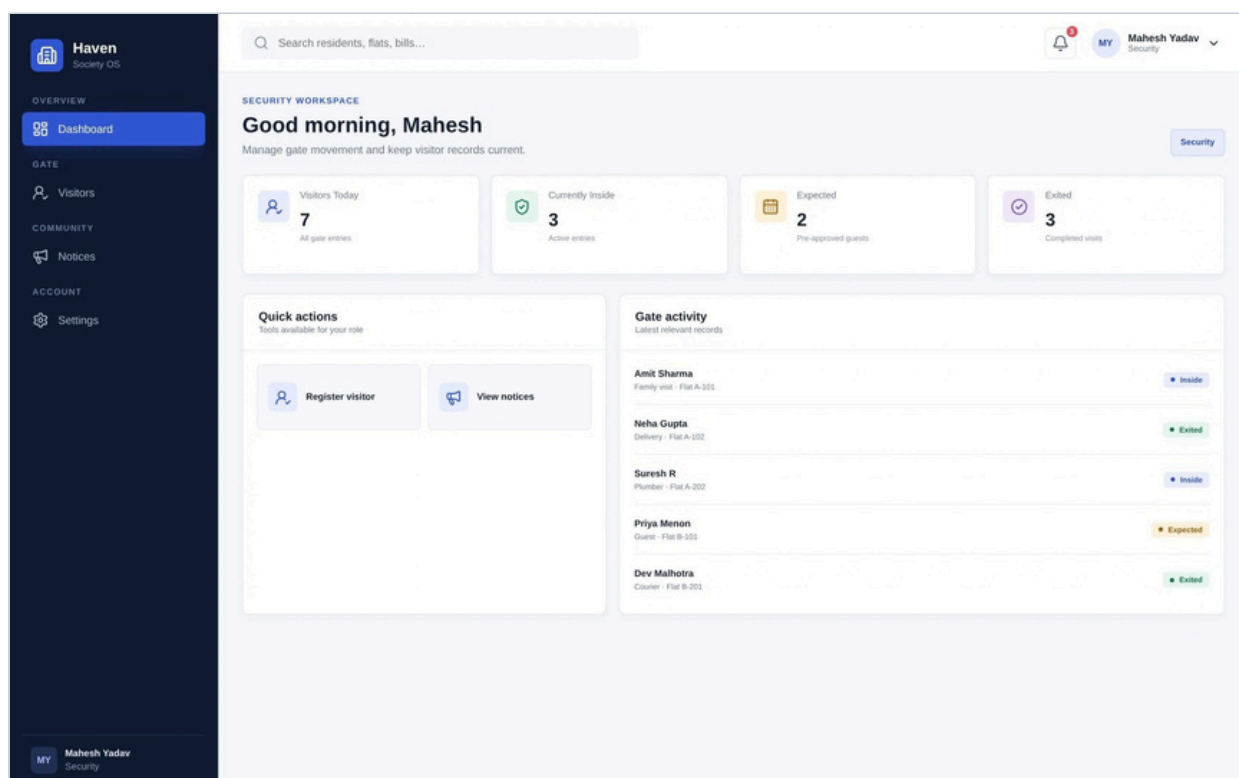


Figure A.19 – Security dashboard

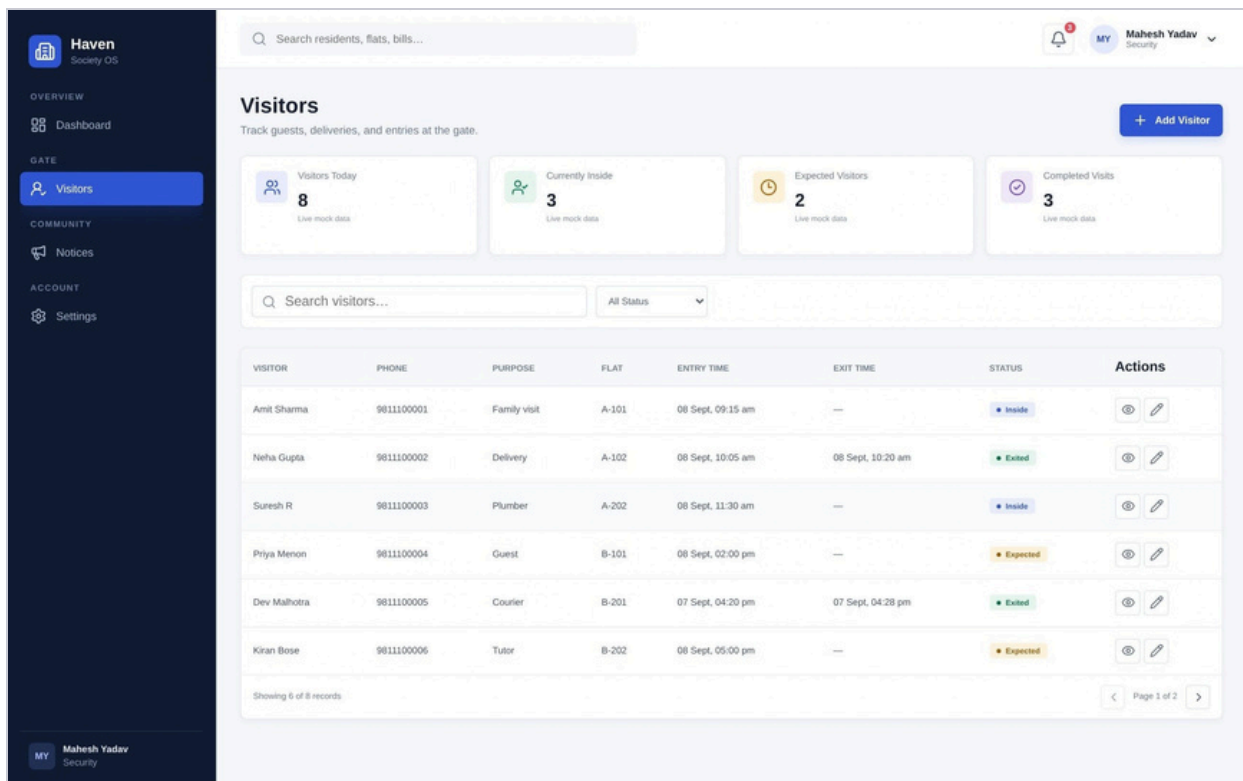


Figure A.20 – Security – visitor management

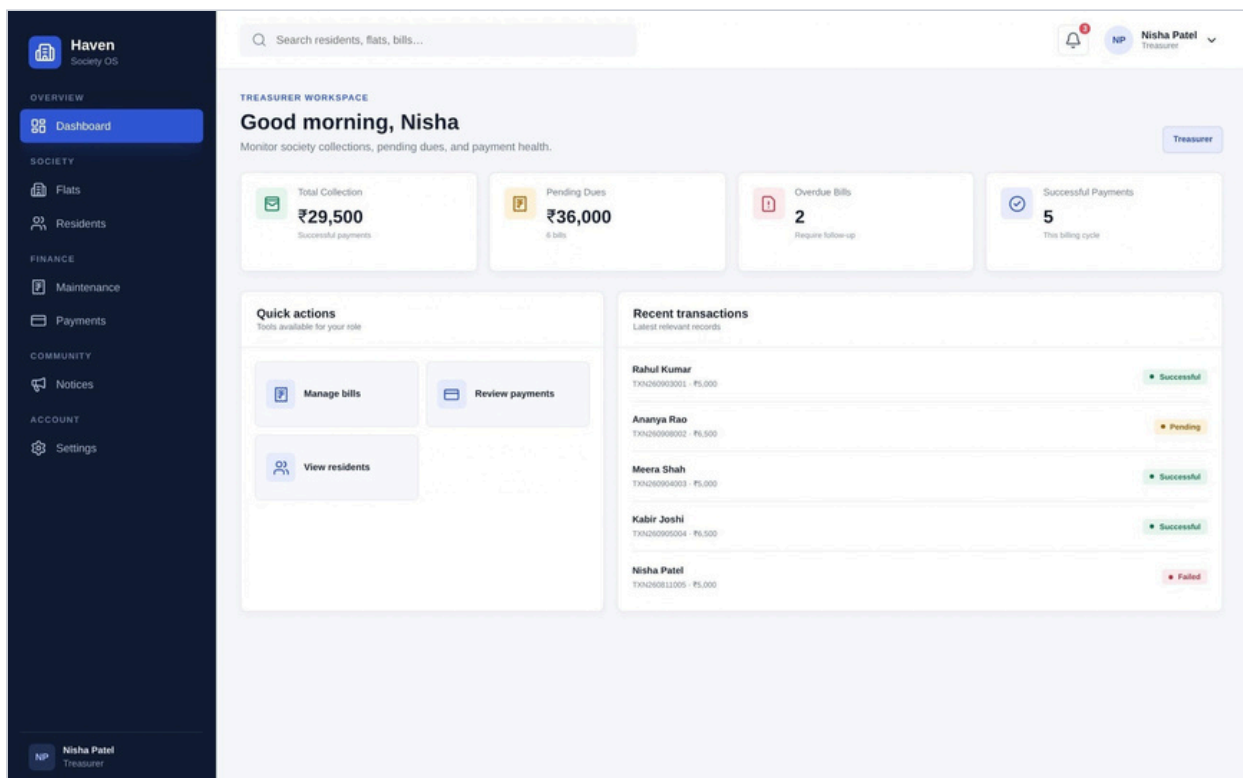


Figure A.21 – Treasurer dashboard

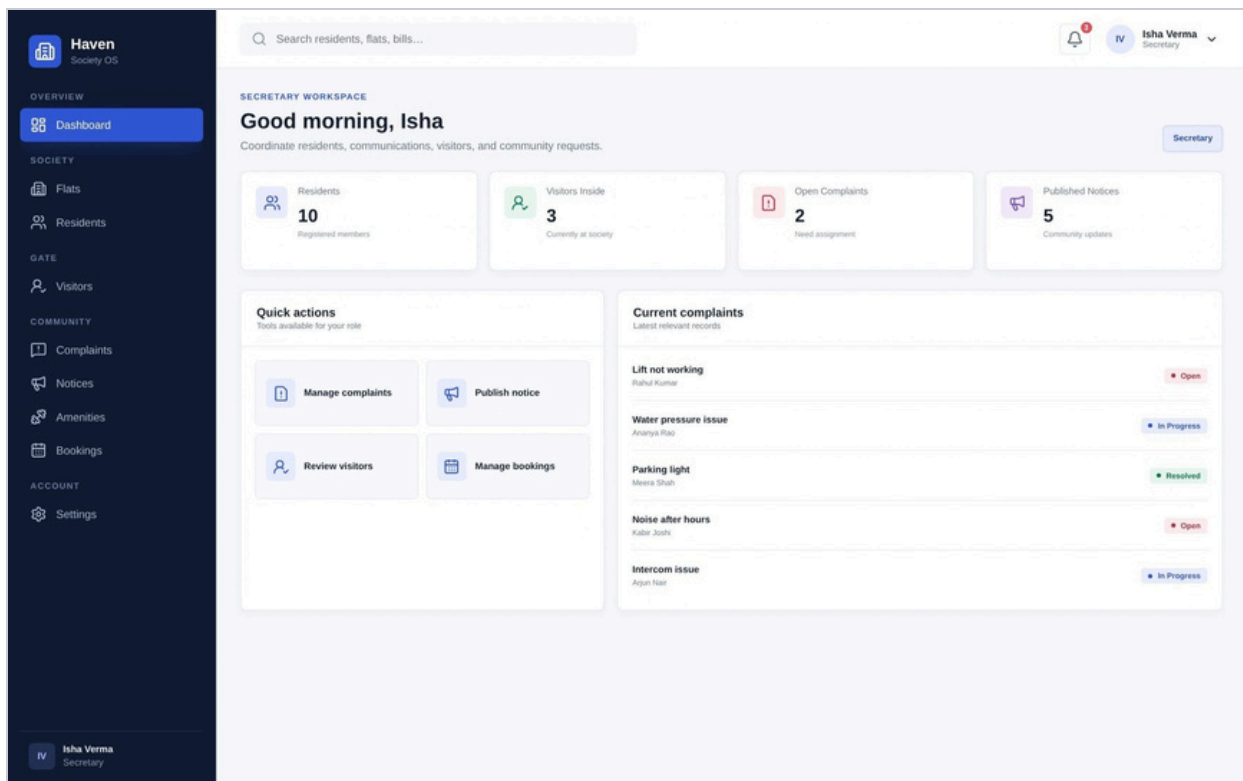


Figure A.22 – Secretary dashboard

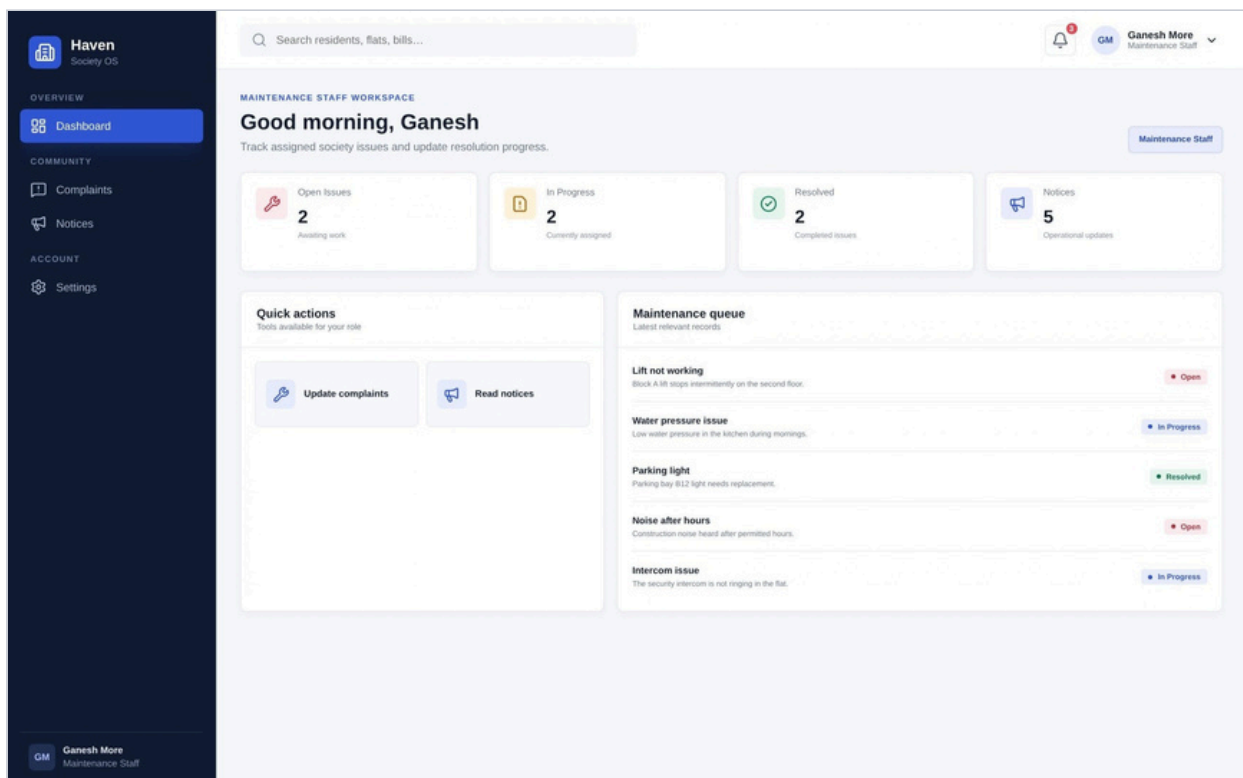


Figure A.23 – Maintenance staff dashboard

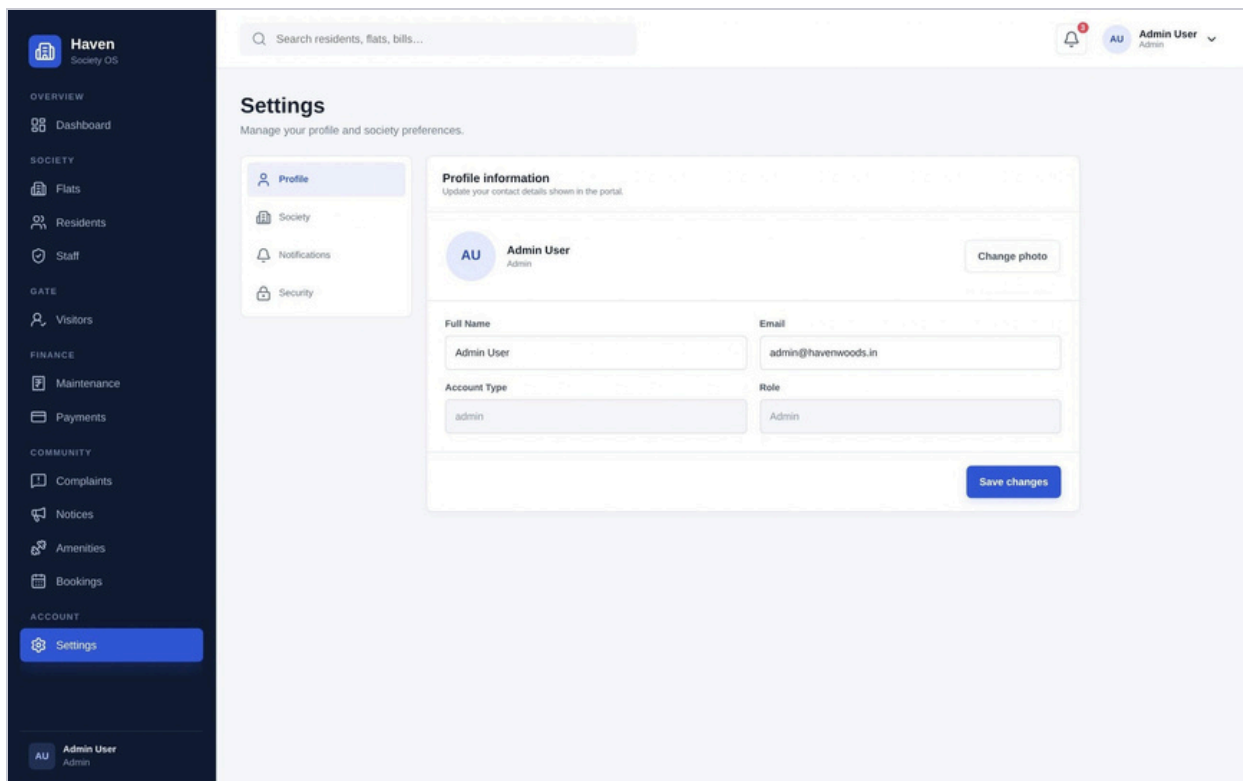


Figure A.24 – Settings / profile page

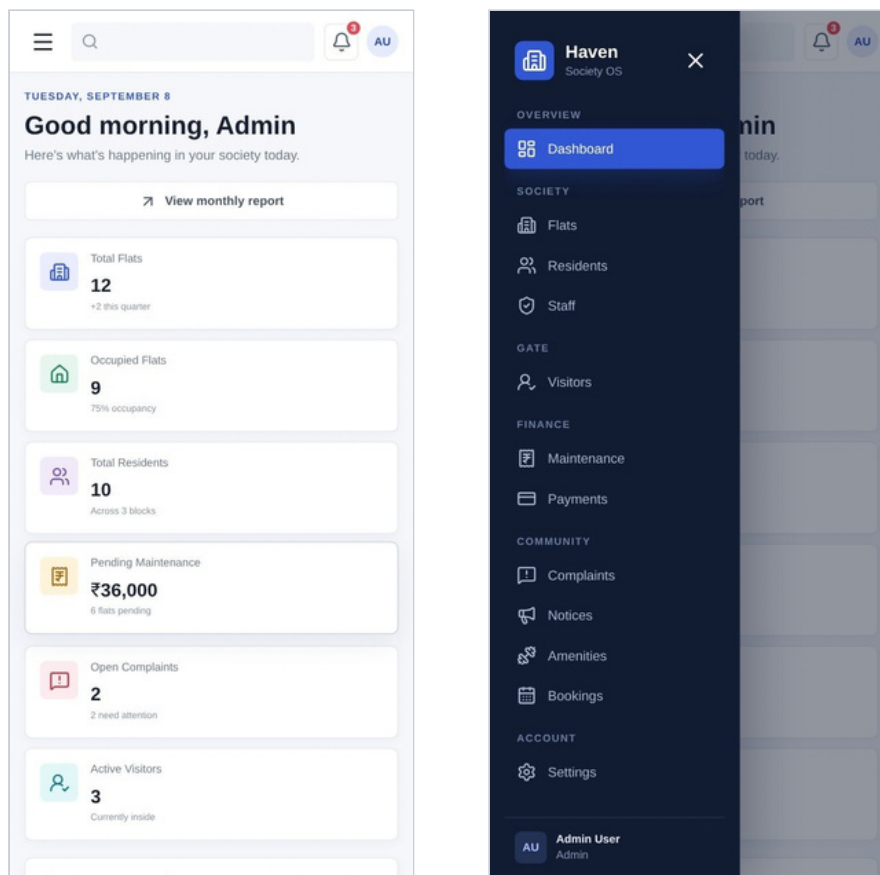


Figure A.25 – Mobile view (390 × 844): dashboard and slide-in navigation menu

## B. Sample Code Snippets

### B.1 Route definitions – *src/App.jsx*

```
function App() {
  return <BrowserRouter><AuthProvider><Routes>
    <Route path="/login" element={<Login />} />
    <Route element={<ProtectedRoute><DashboardLayout /></ProtectedRoute>}>
      <Route index element={<Dashboard />} />
      <Route path="flats" element={<Flats />} />
      <Route path="residents" element={<Residents />} /> <Route
      path="visitors" element={<Visitors />} /> <Route path="staff"
      element={<Staff />} />
      <Route path="maintenance" element={<Maintenance />} />
      <Route path="payments" element={<Payments />} /> <Route
      path="complaints" element={<Complaints />} /> <Route
      path="notices" element={<Notices />} />
      <Route path="amenities" element={<Amenities />} /> <Route
      path="bookings" element={<Bookings />} /> <Route
      path="settings" element={<Settings />} />
    </Route>
    <Route path="*" element={<Navigate to="/" replace />} />
  </Routes></AuthProvider></BrowserRouter>
}
```

### B.2 Route guard – *src/components/ProtectedRoute.jsx*

```
export default function ProtectedRoute({ children }) {
  const location = useLocation()
  const user = getCurrentUser()
  if (!user) return <Navigate to="/login" replace />
  if (!canAccessPath(user, location.pathname)) return <Navigate to=
  {firstAllowedPath(user)} replace />
  return children
}
```

### ***B.3 Role permissions – src/config/permissions.js (excerpt)***

```
const accessByRole = {
  Admin: ['/', '/flats', '/residents', '/visitors', '/staff', '/maintenance',
    '/payments',
    '/complaints', '/notices', '/amenities', '/bookings', '/settings'],
  Treasurer: ['/', '/flats', '/residents', '/maintenance', '/payments', '/notices',
    '/settings'],
  Resident: ['/', '/visitors', '/maintenance', '/payments', '/complaints', '/notices',
    '/amenities', '/bookings', '/settings'],
  Security: ['/', '/visitors', '/notices', '/settings'],
  ...
}
const manageByRole = {
  Admin: ['flats', 'residents', 'visitors', 'staff', 'maintenance', 'payments',
    'complaints', 'notices', 'amenities', 'bookings'],
  Treasurer: ['maintenance', 'payments'],
  Security: ['visitors'],
  'Maintenance Staff': ['complaints'],
  ...
}
const createByRole = { Resident: ['visitors', 'complaints', 'bookings'] }

export function canEditRecord(user, resource, record) {
  if (canManageResource(user, resource)) return true
  return user?.role === 'Resident' && ['visitors', 'complaints',
    'bookings'].includes(resource)
    && ownsRecord(user, resource, record)
}
```

### ***B.4 Declarative module configuration – src/config/resources.js (excerpt)***

```
maintenance: {
  title: 'Maintenance', subtitle: 'Track monthly bills and outstanding society dues.', idField: 'bill_id',
  itemName: 'Bill',
  columns: [['bill_id', 'Bill ID'], ['flat_id', 'Flat', 'flat'], ['bill_month', 'Month'],

  fields: [
    ['amount', 'Amount', 'currency'], ['due_date', 'Due Date', 'date'],
    ['bill_status', 'Status', 'badge']],

    { name: 'flat_id', label: 'Flat', type: 'flat', required: true },
    { name: 'bill_month', label: 'Bill Month', required: true, placeholder: 'September
2026' },
    { name: 'amount', label: 'Amount', type: 'number', required: true },
    { name: 'due_date', label: 'Due Date', type: 'date', required: true },
    { name: 'bill_status', label: 'Status', type: 'select',
      options: select(['Paid', 'Pending', 'Overdue']), required: true },
  ],
  stats: [['Total Bills', 'all'], ['Paid', 'Paid'], ['Pending', 'Pending'], ['Overdue', 'Overdue']],
  statusField: 'bill_status',
},
```

### ***B.5 A complete module page – src/pages/Payments.jsx***

```
import ResourcePage from './ResourcePage'
export default function Payments() { return <ResourcePage resource="payments" /> }
```

### ***B.6 Relationship rendering in the data table – src/components/DataTable.jsx (excerpt)***

```
const render = (row, key, type) => {
  const value = row[key]
  if (type === 'badge') return <StatusBadge value={value} />
  if (type === 'currency') return currency.format(value)
  if (type === 'date') return date(value)
  if (type === 'flat') {
    const flat = data.flats.find((item) => Number(item.flat_id) === Number(value))
    return flat ? `${flat.block_name}-${flat.flat_number}` : '—'
  }
  if (type === 'resident')
    return data.residents.find((item) => Number(item.resident_id) ===
    Number(value)).resident_name || '—'
  ...
}
```

## C. Installation / Setup Instructions

### Prerequisites

- Node.js 20 LTS or later, which includes npm. Check with `node -v` and `npm -v`.
- Git (optional, for cloning the repository).
- A modern browser such as Google Chrome, Microsoft Edge, or Mozilla Firefox.
- Visual Studio Code or another code editor (optional, for viewing and editing the code).

### Steps

1. **Get the source code.** Clone the repository with `git clone <repository-url>`, or download the project ZIP file and extract it. Open a terminal in the `society-management-frontend` folder.
2. **Install dependencies.** Run `npm install`. This downloads React, Vite, React Router, Recharts, Lucide React, and ESLint into the `node_modules` folder.
3. **Start the development server.** Run `npm run dev`. Vite prints a local address, normally `http://localhost:5173`.
4. **Open the application.** Open the address in a browser. The login page appears.
5. **Sign in.** Choose a role from the *Demo Role* list (the credentials are filled in automatically) and click *Sign in*. The demo accounts are:

| Role              | Person       | Email                      | Password    |
|-------------------|--------------|----------------------------|-------------|
| Admin Committee   | Admin User   | admin@havenwoods.in        | admin123    |
| Treasurer         | Vikram Singh | vikram@example.com         | demo123     |
| Secretary         | Nisha Patel  | nisha@example.com          | demo123     |
| Resident          | Isha Verma   | isha@example.com           | demo123     |
| Security          | Rahul Kumar  | rahul@gmail.com            | rahul123    |
| Housekeeping      | Mahesh Yadav | security@havenwoods.in     | security123 |
| Maintenance Staff | Lata Pawar   | housekeeping@havenwoods.in | staff123    |
|                   | Ganesh More  | maintenance@havenwoods.in  | staff123    |

### Other commands

| Command                    | Purpose                                                      |
|----------------------------|--------------------------------------------------------------|
| <code>npm run lint</code>  | Check the code with ESLint.                                  |
| <code>npm run build</code> | Create an optimized production build in <code>dist/</code> . |

|                 |                                                       |
|-----------------|-------------------------------------------------------|
| npm run preview | Serve the production build locally for a final check. |
|-----------------|-------------------------------------------------------|

**Troubleshooting.** If port 5173 is busy, Vite automatically uses the next free port and prints it. If npm install fails, confirm the Node.js version and delete node\_modules and package-lock.json only as a last resort before reinstalling. Data changes are kept in memory, so refreshing the browser restores the sample data; this is expected in the current release.

## D. User Manual / Guide

### *Signing in and out*

Open the portal, choose your role or enter your email and password, and click *Sign in*. To sign out, click your name in the top-right corner and choose *Logout*.

### *Finding your way around*

- The **sidebar** on the left lists the modules available to your role. On a phone, tap the menu icon (≡) at the top-left to open it.
- The **search bar** at the top finds residents, flats, visitors, complaints, bills, and notices. Type at least two letters and click a result to open its page.
- The **bell icon** shows recent notifications; the **profile menu** opens Settings or logs you out.

### *Working with records (all module pages)*

1. Use the search box above the table to find records, and the dropdown filters to narrow them by block, status, or type.
2. Click a column heading to sort; click again to reverse the order. Use the arrows at the bottom to move between pages.
3. Click **Add ...** to create a record. Fields marked with a red asterisk are required. Choose related items (flat, resident, bill, amenity) from the dropdowns.
4. Click the **eye** icon (or the row) to view details, the **pencil** icon to edit, and the **bin** icon to delete. Deleting asks for confirmation.
5. A green message at the bottom of the screen confirms each successful action.

### *Guide by role*

| Role      | Common tasks                                                                                                 |
|-----------|--------------------------------------------------------------------------------------------------------------|
| Admin     | Review the dashboard; add flats, residents, and staff; manage every module.                                  |
| Committee | Monitor open complaints and upcoming bookings; manage residents, visitors, notices, amenities, and bookings. |

|                   |                                                                                                                                                                                                      |
|-------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Treasurer         | Create monthly bills in <i>Maintenance</i> ; record payments in <i>Payments</i> by choosing the bill and resident and entering the method and transaction ID; follow overdue bills on the dashboard. |
| Secretary         | Publish notices; update complaint status; review visitors and bookings.                                                                                                                              |
| Resident          | Check <i>Amount Due</i> on the dashboard; view bills and payments; use <i>Raise complaint</i> , <i>Register visitor</i> , and <i>Book amenity</i> ; read notices.                                    |
| Security          | Add each arriving visitor with status <i>Inside</i> ; edit the entry with the exit time and status <i>Exited</i> when they leave; check <i>Expected</i> visitors.                                    |
| Maintenance Staff | Open <i>Complaints</i> , read the description, and change status to <i>In Progress</i> and then <i>Resolved</i> .                                                                                    |
| Housekeeping      | Read the latest notices on the dashboard and the Notices page.                                                                                                                                       |

### **E. Geo Tag Photos with Guide**

*Insert the team's geo-tagged photograph(s) taken with the project guide here.*

## F. Review Forms with Guide Signatures

*Insert the scanned review / rubrics evaluation form(s) for this project, signed by the guide, here.*